# Project 1: SIMD Advantage Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 21, 2026

**Abstract**

We quantify scalar-versus-SIMD speedup for four single-threaded numeric kernels (SAXPY, dot product, elementwise multiply and a 3-point stencil) on an Intel Core i9-10900F (Comet Lake, AVX2+FMA, 256 bit vectors) running at a fixed 2.8 GHz. With cache-resident data a `-march=native` build reaches about 5.8× over scalar for SAXPY (8 float32 lanes), while the generic auto-vectorized (SSE, 128 bit) build reaches about 3×; once the working set spills to DRAM the gain compresses to 1.88–2.02× for SAXPY (1.5–1.6× for elementwise multiply, 2.6–2.8× for the stencil; ranges span $N = 2^{24}$–$2^{25}$ and up to five separate sweeps). There the native kernels sit *on* the measured DRAM bandwidth roof, and the remaining speedup only reflects that scalar SAXPY needs ≈ 1.5–2.3 cycles of core work per element (at an IPC of 3–4.6), more than memory needs at this clock. Float64 (4 lanes) usually keeps about half of float32's cache-resident gain, but not uniformly (stencil at $N = 16\,384$: 3.02× versus 3.44×), and operation cost matters as much as lane count: without SSE4.1's `pmulld` the SSE2 build gains only 1.86× on int32 multiply versus 3.53× on float32. Non-unit stride or gather access erases the advantage; an explicit AVX2 hardware gather (`vgatherdps`) is even 1.5–1.6× *slower* than the scalar gather in cache. Our negative case is memory-bound SAXPY: the loop provably vectorizes, yet wider vectors buy nothing in DRAM; a second anomaly is the dot product, whose non-reassociated reduction leaves a 4-cycle scalar dependency chain and ≈ 1× speedup. A cache-aware roofline whose ceilings are measured by separate microbenchmarks—an FMA-throughput loop (89.2 GFLOP/s float32, 32.0 FLOP/cycle) and a STREAM-style triad per cache level (from 156 GiB/s in L1 down to 24.3 GiB/s from DRAM)—places L3- and DRAM-resident native SAXPY, multiply and stencil at 95 % to 107 % of their bandwidth roof (up to 120 % for SAXPY at one intermediate size; the triad roof is a reference, not a hard bound).

## 1 Introduction

Single-instruction-multiple-data (SIMD) units let one instruction operate on a whole vector of elements. On the Intel Core i9-10900F studied here the widest available unit is AVX2 with FMA, a 256 bit register that holds 8 float32 or 4 float64 lanes, so the *ideal* speedup over scalar code is 8× (float32) or 4× (float64). Real kernels rarely reach that ceiling: loads and stores, loop prologue/epilogue handling, reduction dependencies, and—above all—finite memory bandwidth cap the benefit. The purpose of this report is to measure where the benefit appears, by how much it falls short of the lane count, and exactly when it disappears.

This report answers:

1. How much faster are auto-vectorized builds than scalar builds for each kernel, and how does that change from cache-resident to DRAM-resident working sets? (Sections 5 and 6)

Table 1: Measurement environment.

| | |
|---|---|
| CPU | Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64) |
| SIMD ISA | sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm) |
| Topology | 1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s) |
| SMT | off |
| Caches | L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance) |
| Memory | 63 GiB total; DIMMs: 2 × 32 GiB DDR4-3200, one per channel (A, B), running at 2933 MT/s (dmidecode) |
| Pages | base 4096 bytes; THP: always |
| OS / kernel | Arch Linux; 7.2.9-arch1-1 |
| Frequency policy | governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance |
| Pinning / isolation | Each run is pinned to one physical core (`--cpu 2`, `sched_setaffinity`); SMT is disabled, so the core has no sibling thread, and no other pinning/isolation is used.; no isolcpus |
| Compilers | gcc (GCC) 16.2.1 20260810; clang version 23.1.3 |
| Tools | perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present |
| GPU (optional) | NVIDIA GeForce RTX 3070; CUDA not available (no CUDA) |
| Env. snapshot | 2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`) |

2. What do alignment, loop tails, stride and gather access, and data type cost? (Sections 7 to 9)

3. Where do these kernels sit in a roofline model of this machine? (Section 10)

4. When does the expected SIMD benefit fail to appear? (Section 11)

In brief: SIMD helps most when data fits in L1/L2, where `native` gives roughly 5.8× for SAXPY and the cache feeds the vector units fast enough to approach the lane count; the advantage shrinks step-by-step as the working set crosses L2 and L3 and falls to 1.88–2.02× for SAXPY in DRAM (1.5–1.6× multiply, 2.6–2.8× stencil), where the vector build runs at the memory-bandwidth roof and only the scalar build's own instruction cost keeps the ratio above 1×. Data type matters through lane count (float64 usually keeps about half the gain) and through the cost of the vector operation (SSE2 int32 multiply), and access pattern matters even more (any stride $\geq 2$ removes the SIMD advantage; gather, even with AVX2 `vgatherdps`, never gains). Two cases show the limits of "it vectorized, so it must be faster": memory-bound SAXPY (Section 11) and the dot-product reduction (Section 4).

# 2 Methodology

## 2.1 Platform

This CPU (Comet Lake, client Skylake core) implements SSE/AVX/AVX2 and FMA but *not* AVX-512, so the widest usable vector register is 256 bit (`ymm`): 8 lanes of float32 or 4 of float64. With `-march=native` GCC targets AVX2+FMA and prefers 256 bit vectors; the default (generic `x86-64`) target only guarantees SSE2, so the plain auto-vectorized build emits 128 bit (`xmm`) code. That 256-vs-128-bit gap is the main reason we keep `autovec` and `native` as separate variants (Table 3).

Table 2: Kernels and floating-point operations (FLOPs) per processed element.

| Kernel | Definition | FLOPs / element |
|---|---|---|
| SAXPY | $y_i \leftarrow a\,x_i + y_i$ | 2 |
| Dot product | $s \leftarrow \sum_i x_i\,y_i$ | 2 |
| Elementwise multiply | $z_i \leftarrow x_i\,y_i$ | 1 |
| 3-point stencil | $y_i \leftarrow a\,x_{i-1} + b\,x_i + c\,x_{i+1}$ | 5 |

Table 3: Build variants. Only the kernel translation unit (`src/kernels.c`) differs; the timing harness (`main.c`, `common.c`) and the reference (`reference.c`) are compiled identically across variants.

| Variant | Kernel compiler flags (from `simd_bench --info`) |
|---|---|
| scalar | `-O3 -fno-tree-vectorize -fno-tree-slp-vectorize` |
| autovec | `-O3` (generic `x86-64` ⇒ SSE2, 128 bit) |
| native | `-O3 -march=native` (AVX2+FMA, 256 bit) |
| native512 | not available on this CPU (no AVX-512; 256 bit is the widest ISA) |

## 2.2 Kernels

We count a multiply and an add as one FLOP each, so a fused multiply–add is two FLOPs; SAXPY and the dot product are therefore 2 FLOP/element (one FMA), elementwise multiply is 1, and the stencil is 5 (three multiplies, two adds). All four kernels from the project brief are implemented and validated; SAXPY is used as the representative kernel for the locality, alignment and stride studies because it has the simplest unit-stride access and a well-known arithmetic intensity.

## 2.3 Build variants

The primary compiler is GCC gcc (GCC) 16.2.1 20260810. The reference implementation additionally uses `-O2 -fno-tree-vectorize -ffp-contract=off` so it evaluates each expression in strict, non-fused order and stays a trustworthy oracle. We deliberately do *not* pass `-ffast-math` to any variant: this keeps float results reproducible but, as Sections 4 and 11 show, it also prevents the compiler from reassociating the dot-product reduction—an effect we wanted to measure rather than hide.

## 2.4 Data collection and hypotheses

All measurements were collected on **2026-10-21** on an idle machine, with the settings checked before each run (governor `performance`, turbo off, SMT off, pinned to CPU 2) and runs serialized with other benchmark jobs through a lock file: `baseline` at 20:38 and the other six experiments back to back from 20:47 to 20:54 (raw directories in Table 11). Before this session the user masked an ACPI general-purpose-event storm (GPE 0x42) that the companion Cache & Memory project's wall-clock check had found stealing $\approx 13\,\%$ of wall time; this root-only setting does not survive a reboot (Section A). Two shared-core checks guard the reported data: no configuration varies by more than 1.36× between its fastest and slowest repetition (a single DRAM-resident alignment configuration; all others $\leq 1.32\times$), and the user-space cycle counter, which would show a shared core as a lower apparent clock, reads 2.78 GHz to 2.79 GHz for every counted configuration.

Each experiment section opens with a hypothesis derived from first principles (lane counts, cache sizes, port counts). Some were drafted after an earlier, superseded turbo-clock sweep and may have been influenced by it, so we present them only as predictions compared against the data, and we

state where the data contradicted them.

### 2.5 Timing and statistics

- Timer: `clock_gettime(CLOCK_MONOTONIC_RAW)` (resolution 1 ns on this host, from `simd_bench --info`). Each repetition times a calibrated batch of kernel invocations lasting at least 25 ms, i.e. roughly $2.5 \times 10^7$ timer ticks, so timer resolution and call overhead are negligible.
- Warm-up: 3 untimed invocations per configuration; repetitions: 12 per configuration.
- Reported statistic: median; error bars show the interquartile range (IQR) across the 12 repetitions. The median resists an occasional scheduler outlier better than the mean. Speedup error bars span the range implied by the two IQRs, $[\, q_{25}(\text{SIMD})/q_{75}(\text{scalar}),\ q_{75}(\text{SIMD})/q_{25}(\text{scalar}) \,]$ (`scripts/speedup_iqr.awk`), a conservative bound.
- Pinning, SMT, governor and turbo: Table 1. For the reported runs the governor is performance, turbo is disabled and SMT is off, which fixes the core at its 2.8 GHz base clock. We do not just assume the clock: the cycle counter (Section 2.6) gives a per-configuration median of 2.78 GHz to 2.79 GHz for all 96 locality and 123 ceilings configurations. The clock is user-mode cycles over wall time, so stolen time would show up as a lower value.
- Effect of the fixed clock: an earlier sweep (2026-10-12, governor `powersave`, turbo on) is not used for any result; its data is no longer in the working tree but remains in the repository history (commit `73f8fb8`, raw directory `20261012-182813`). There, every cache-resident rate at $N = 1024$ was 1.72–1.75× the fixed-clock value (dot product: 2.43 GFLOP/s to 2.49 GFLOP/s in cache at its latency bound of 0.5 FLOP/cycle, i.e. ≈ 4.9 GHz), while DRAM bandwidth does not scale with the clock. Its DRAM-resident SAXPY speedups were therefore smaller, 1.32–1.44× ($N \geq 2^{24}$, baseline, locality and negative-case sweeps); this is the single turbo figure we quote (Section 6).
- Every kernel result is written through a `volatile` sink (`acs_consume_*`) and the kernels live in their own translation unit, so no loop is eliminated as dead code.
- Configuration order: the sweep is run as nested loops over size and variant (`scripts/run.sh`); it is not shuffled, which we note as a limitation.

### 2.6 Machine ceilings and hardware counters

The roofline (Section 10) needs a peak FLOP rate and memory bandwidths that are *not* derived from the kernels it is used to explain. A separate program, `build/ceilings/ceilings` (`src/ceilings.c`, `run.sh ceilings`), measures them:

- *Peak FLOP rate*: a register-only loop of 12 independent accumulators updated as $a \leftarrow a\,m + c$. FMA (and multiply/add) latency is 4 cycles on this core and two ports execute them, so at least $4 \times 2 = 8$ independent chains are needed to saturate both ports; 12 leaves headroom while staying in registers. Variants: scalar `mulss`+`addss` (the ceiling of our `scalar` build), scalar FMA, SSE `mulps`+`addps` on `xmm` (the ceiling of `autovec`, which has no FMA), and AVX2 `vfmadd231ps`/`pd` on `ymm` (the ceiling of `native`).

- *Bandwidth*: STREAM-style *read* ($s \mathrel{+}= a_i$), *copy* ($b_i = a_i$) and *triad* ($a_i = b_i + s\,c_i$) written with AVX2 loads/stores, at a total footprint inside each level (L1 16 KiB, L2 128 KiB, L3 8 MiB, DRAM 512 MiB) and over a fine footprint sweep from 4 KiB to 512 MiB. Bytes are counted STREAM-style (explicit loads + stores) and, separately, including the write-allocate read that a store to a line not in L1 costs. The roofline uses the *actual* traffic of each level: explicit bytes in L1, where every store hits an already-cached line, and bytes including write-allocate in L2, L3 and DRAM; kernel intensities are counted the same way (Table 10). The read stream keeps only 4 accumulators, so its 4-cycle add chain caps it at one 32 B load per cycle (89 GiB/s): in L1 and L2 it measures that latency, not the cache, and we never use it as a roof.

*Hardware counters.* The AI agent that ran the benchmarks had no root. The user applied the root-only settings: governor, turbo, SMT, the GPE mask, and `perf_event_paranoid`, which was 0 for the reported session. The SIMD benchmark deliberately counts only *core-PMU* events of its own thread with the kernel excluded, which works unprivileged up to `perf_event_paranoid` = 2. Doing so from inside the benchmark lets us bracket exactly the timed repetitions. `src/perfctr.c` therefore calls `perf_event_open(2)` directly for cycles, instructions, LLC misses and L1D read misses (kernel time excluded), enabled only around each timed repetition; an event that cannot be opened is reported as `NA` instead of failing. The locality sweep records them (Section 6) and the ceilings record cycles, from which the effective clock and FLOP/cycle follow.

### 2.7 Data initialization and working-set sizes

Inputs are filled with a deterministic linear-congruential sequence in $[-0.5, 0.5)$ (fixed seed), which avoids denormals, NaNs and overflow; SAXPY's in-place update grows $y$ only linearly in the repetition count, so it never overflows float32. The gather pattern uses a Fisher–Yates permutation of $[0, N)$ so every element is touched exactly once with no locality.

Each kernel streams two or three arrays, so the bytes touched are (arrays)×(element size)×$N$. For float32 SAXPY (two arrays, 8 B/element) the working set crosses this machine's 32 KiB L1d near $N \approx 4096$, the 256 KiB L2 near $N \approx 32\,768$, and the 20 MiB L3 near $N \approx 2.6 \times 10^6$; beyond that it is DRAM-resident. These boundaries are annotated on the locality plot (Section 6).

## 3 Correctness Validation

Every build variant is checked two independent ways before any of its timings are trusted (`make test`, `scripts/validate.sh`).

*Reference comparison.* `src/reference.c` contains plain scalar loops compiled with `-fno-tree-vectorize -ffp-contract=off`, so it evaluates each kernel in strict, non-fused, left-to-right order. The driver runs the kernel under test and the reference on identical inputs and compares element-by-element (or the scalar result, for the dot product) before timing.

| | |
|---|---|
| **Cases** | kernels × {float32, float64}; $N$ around multiples of the lane count $(1, 2, 3, 7, 8, 9, 15, 16, 17, 31, 33, 64, 127, 1000, 1024, 4095, 4096, 4097, 65536)$; start offsets 4, 8, 12, 16, 20, 28, 32 and 60 B; strides $2, 4, 8, 16$; gather; explicit AVX2 gather (native only, $N \in \{7, 8, 9, 16, 17, 1000, 4096, 65536\}$); int32 multiply |
| **Tolerance** | relative error $\le 10^{-5}$ (float32) and $\le 10^{-12}$ (float64). The bound is loose enough for the one legal source of disagreement—FMA contraction in the vectorized builds versus the non-fused reference—and tight enough to catch real bugs. |
| **Command** | `make test` |

*Known-answer self-test* (`simd_bench --selftest`), independent of the reference: the dot product of two all-ones vectors equals $N$; $y = 2x + y$ with $x = 1, y = 3$ gives 5 everywhere; $z = x\,y$ with

$x = 2, y = 3$ gives 6; and the stencil with $a = b = c = 1$ on a constant-1 input gives 3 in the interior; and the explicit AVX2 gather through a reversed permutation gives 5 everywhere. These guard against a reference and kernel sharing the same mistake.

The sweep covers sizes with and without a vector-width remainder (to exercise the vector epilogue and scalar tail) and deliberately misaligned start addresses (to exercise the loop prologue), so correctness is established precisely in the regimes the later experiments stress.

Result (`data/raw/validation/20261021-231629/make_test.txt`): all 578 cases pass (190 each for scalar and autovec, 198 for native, which adds the AVX2 gather). Each passing comparison logs its maximum relative error. The largest is $2.1 \times 10^{-7}$ (native float32 dot product, $N = 127$); every other case, including all float64 and int32 cases, matches the reference bit for bit. The test coefficients ($a = 0.5$; stencil $0.25, 0.5, 0.25$) make every product exact, so FMA contraction cannot change SAXPY or stencil results here, and only the reduction exercises the tolerance.

# 4 Vectorization Verification

**Hypothesis (from first principles, then compared with the data)**

The `scalar` build should emit only scalar SSE (`ss`/`sd`) instructions and report zero vectorized loops. `autovec`, limited to the generic SSE2 target, should vectorize the streaming kernels at 128 bit (4 float32 lanes). `native` should vectorize them at 256 bit (`ymm`, 8 float32 / 4 float64 lanes) and fuse the multiply–add in SAXPY and the stencil into `vfmadd`. We expect the compiler to *refuse* to vectorize the dot-product reduction usefully, because summation reassociation is illegal without `-ffast-math`.

Table 4 counts, for each kernel function and variant, the instructions that operate on vector registers or use packed operations, extracted by `objdump` in `make report`. (A few "SIMD" matches in the scalar column are register-clearing `pxor`/`movaps` on `xmm`, not vectorized loop bodies—the compiler reports *zero* vectorized loops for `scalar`.)

## 4.1 Compiler report

The evidence below (`data/raw/vectorization/20261021-001032`) comes from `make report`, which recompiles `src/kernels.c` with each variant's exact flags and the same GCC as the timed builds. `kernels.c` has not changed since. We checked with `objdump` that every kernel function there has the same instruction sequence as in the linked `simd_bench` binaries that produced the measurements, differing only in absolute addresses and alignment padding. `make report` does not record an `env.txt`; the environment is that of the timed runs (Table 1).

GCC's `-fopt-info-vec` confirms the widths. For SAXPY (`kernels.c:29`):

```
# native  (-O3 -march=native):
src/kernels.c:29:26: optimized: loop vectorized using 32 byte vectors and unroll factor 8
src/kernels.c:29:26: optimized: epilogue loop vectorized using 16 byte vectors and unroll factor 4
# autovec (-O3, generic x86-64):
src/kernels.c:29:26: optimized: loop vectorized using 16 byte vectors and unroll factor 4
src/kernels.c:29:26: optimized: epilogue loop vectorized using 8 byte vectors and unroll factor 2
# scalar  (-O3 -fno-tree-vectorize): 0 loops vectorized
```

32 B vectors = 256 bit = 8 float32 lanes (native); 16 B = 128 bit = 4 lanes (autovec). GCC's "unroll factor" is the vectorization factor (elements per iteration), not extra unrolling: the disassembly below has one `ymm` FMA per iteration. In the disassembly the native "epilogue loop" is a single 128 bit step, taken when 4 or more elements remain, followed by up to three scalar iterations; autovec's is a single 8 B (2-lane) step followed by at most one scalar iteration. The loop is *versioned* for possible

Table 4: SIMD instructions per kernel function and build variant.

| Variant | Function | Instructions | SIMD instructions | Widest register |
|---|---|---|---|---|
| scalar | saxpy_f32 | 15 | 1 | xmm(128) |
| scalar | dot_f32 | 19 | 4 | xmm(128) |
| scalar | mul_f32 | 12 | 0 | scalar |
| scalar | stencil_f32 | 34 | 1 | xmm(128) |
| scalar | saxpy_f64 | 14 | 1 | xmm(128) |
| scalar | dot_f64 | 19 | 4 | xmm(128) |
| scalar | mul_f64 | 12 | 0 | scalar |
| scalar | stencil_f64 | 31 | 1 | xmm(128) |
| scalar | mul_i32 | 13 | 0 | scalar |
| scalar | saxpy_f32_stride | 31 | 1 | xmm(128) |
| scalar | saxpy_f32_gather | 15 | 1 | xmm(128) |
| scalar | saxpy_f32_gather_avx2 | 14 | 1 | xmm(128) |
| autovec | saxpy_f32 | 67 | 11 | xmm(128) |
| autovec | dot_f32 | 58 | 11 | xmm(128) |
| autovec | mul_f32 | 63 | 6 | xmm(128) |
| autovec | stencil_f32 | 121 | 25 | xmm(128) |
| autovec | saxpy_f64 | 48 | 7 | xmm(128) |
| autovec | dot_f64 | 42 | 7 | xmm(128) |
| autovec | mul_f64 | 48 | 4 | xmm(128) |
| autovec | stencil_f64 | 78 | 16 | xmm(128) |
| autovec | mul_i32 | 69 | 4 | xmm(128) |
| autovec | saxpy_f32_stride | 84 | 12 | xmm(128) |
| autovec | saxpy_f32_gather | 14 | 1 | xmm(128) |
| autovec | saxpy_f32_gather_avx2 | 14 | 1 | xmm(128) |
| native | saxpy_f32 | 78 | 11 | ymm(256) |
| native | dot_f32 | 78 | 19 | ymm(256) |
| native | mul_f32 | 79 | 9 | ymm(256) |
| native | stencil_f32 | 110 | 16 | ymm(256) |
| native | saxpy_f64 | 60 | 11 | ymm(256) |
| native | dot_f64 | 57 | 13 | ymm(256) |
| native | mul_f64 | 69 | 9 | ymm(256) |
| native | stencil_f64 | 88 | 16 | ymm(256) |
| native | mul_i32 | 80 | 9 | ymm(256) |
| native | saxpy_f32_stride | 93 | 12 | ymm(256) |
| native | saxpy_f32_gather | 14 | 1 | xmm(128) |
| native | saxpy_f32_gather_avx2 | 95 | 18 | ymm(256) |

aliasing between `x` and `y` (a run-time overlap check), and GCC does not peel iterations to reach an aligned start.

### 4.2 Disassembly

The native SAXPY inner loop is a single fused multiply–add on `ymm` registers:

```
vmovups   (%rdi,%rax,1),%ymm0           # load 8 x[i]
vfmadd213ps (%rsi,%rax,1),%ymm1,%ymm0   # ymm0 = a*x + y   (one FMA, 8 lanes)
vmovups   %ymm0,(%rsi,%rax,1)           # store 8 y[i]
```

whereas the scalar build does one element at a time with scalar SSE. It cannot fuse the multiply and add because the generic `x86-64` target it compiles for does not include the FMA instruction

set (GCC's default `-ffp-contract=fast` would contract them if FMA were available, as it does in `native`):

```
movss (%rdi,%rax,4),%xmm0  # x[i]
mulss %xmm1,%xmm0          # a*x[i]
addss (%rsi,%rax,4),%xmm0  # + y[i]   (separate mul + add: generic x86-64 has no FMA)
movss %xmm0,(%rsi,%rax,4)  # y[i]
```

The dot product is the instructive exception. Native *does* multiply eight lanes at once, but the horizontal sum is a serial chain of scalar `vaddss`:

```
vmulps (%rdi,%rax,1),%ymm1,%ymm1  # 8 products at once ...
vaddss %xmm1,%xmm0,%xmm0          # ... but summed one lane at a time,
vaddss %xmm3,%xmm0,%xmm0          #     in strict source order
...
```

### 4.3 Stride and gather code

Table 4 credits `saxpy_f32_stride` with `ymm` instructions in `native`, but that is loop *versioning*, not strided SIMD: GCC tests `s == 1` at run time and only that copy of the loop is vectorized (the same `vmovups`/`vfmadd213ps` body as SAXPY). For $s > 1$ the code is a scalar `vfmadd213ss` loop in `native` and `mulss`/`addss` in the other builds. The plain gather loop (`saxpy_f32_gather`) is not vectorized by any build ("no vectype for stmt"), so we added an explicit AVX2 version, `saxpy_f32_gather_avx2`, compiled only into `native`:

```
vmovdqu    (%rax),%ymm1                # 8 indices idx[i..i+7]
vgatherdps %ymm5,(%rdi,%ymm1,4),%ymm0  # gather 8 x[idx]
vgatherdps %ymm7,(%rsi,%ymm1,4),%ymm3  # gather 8 y[idx]
vfmadd132ps %ymm4,%ymm3,%ymm0          # one 8-lane FMA
vmovss     %xmm0,(%rsi,%rdx,4)         # ... then 8 scalar stores:
vextractps $0x1,%xmm0,(%rsi,%rdx,4)    #     AVX2 has no scatter
```

It is what a hand-vectorized gather looks like on this ISA: the loads are SIMD, the stores cannot be.

### 4.4 Interpretation

The evidence matches the hypothesis. `native` uses `ymm` (256 bit) with FMA for SAXPY, multiply, and the stencil; `autovec` uses `xmm` (128 bit); `scalar` is genuinely scalar. Tails are handled by one narrower vector step plus a short scalar remainder (above). There are no AVX-512 mask registers on this CPU; AVX2's masked loads and stores (`vmaskmovps`) could have processed the remainder in one masked step, but GCC does not use them here and we did not try a hand-written masked tail. The dot product shows why "the loop vectorized" is not the same as "the loop is fast": without `-ffast-math` the compiler keeps the reduction's addition order, so the packed multiplies feed a latency-bound scalar accumulator. We quantify the throughput consequence in Sections 5 and 11.

## 5 Scalar vs. SIMD Baseline

**Hypothesis (from first principles, then compared with the data)**

For compute-friendly, unit-stride kernels (SAXPY, elementwise multiply, stencil) we expect the `native` speedup to approach the float32 lane count (8×) while data is L1/L2-resident, and to fall toward 1× once the working set is DRAM-resident and all variants share the same bandwidth. `autovec` (4 lanes) should track at roughly half the `native` speedup. The dot product should barely speed up at any size because its reduction does not reassociate (Section 4).

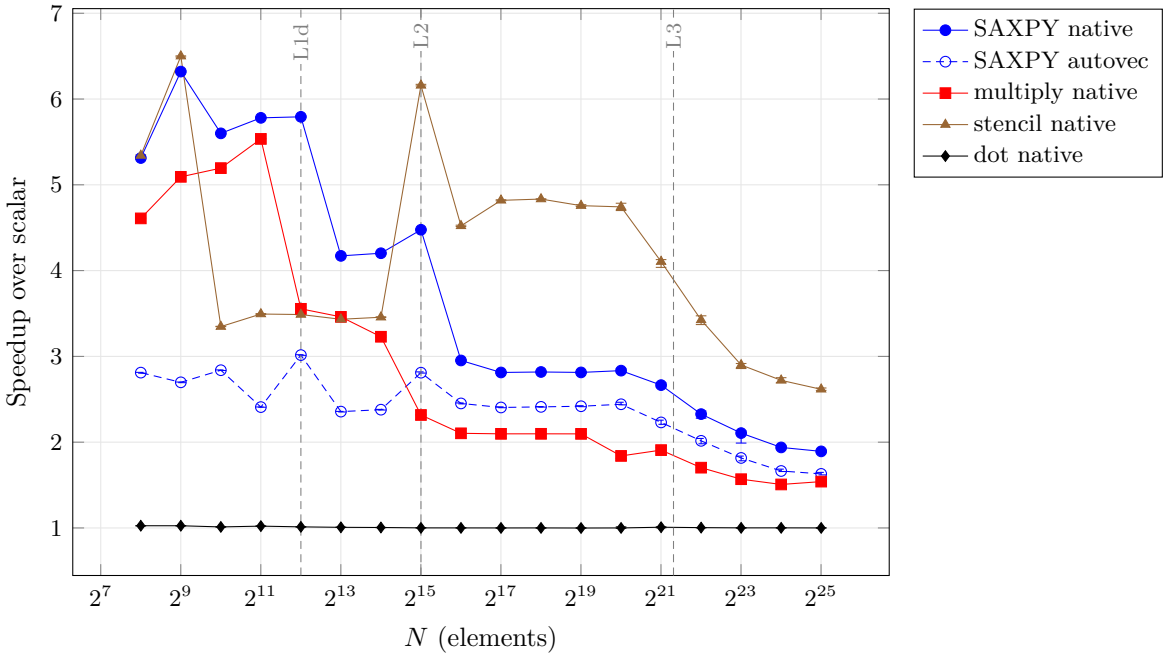



Figure 1: Speedup of the SIMD builds over scalar (median; error bars: range implied by the two IQRs, Section 2). Gains peak while data is cache-resident and compress in DRAM (at $N = 2^{24}$: SAXPY 1.94×, multiply 1.51×, stencil 2.72×); the dot product never benefits.

Table 5: Achieved GFLOP/s (median) of every kernel and build at one $N$ per level: L1 ($N = 1024$), L2 (16 384), L3 (524 288) and DRAM (16 777 216). The same points appear in the roofline (Figure 10).

| Kernel | Build | L1 | L2 | L3 | DRAM |
|---|---|---|---|---|---|
| SAXPY | scalar | 3.59 | 3.32 | 3.08 | 2.40 |
| | autovec | 10.19 | 7.89 | 7.44 | 3.99 |
| | native | 20.10 | 13.94 | 8.66 | 4.65 |
| dot | scalar | 1.41 | 1.39 | 1.39 | 1.33 |
| | autovec | 1.41 | 1.39 | 1.39 | 1.33 |
| | native | 1.42 | 1.39 | 1.39 | 1.33 |
| multiply | scalar | 1.78 | 1.65 | 1.56 | 1.08 |
| | autovec | 6.30 | 3.80 | 3.14 | 1.53 |
| | native | 9.25 | 5.32 | 3.26 | 1.63 |
| stencil | scalar | 4.95 | 4.89 | 4.34 | 4.03 |
| | autovec | 13.59 | 11.83 | 13.12 | 8.72 |
| | native | 16.57 | 16.91 | 20.67 | 10.95 |

| | |
|---|---|
| **Fixed** | float32; unit stride; aligned arrays; one thread pinned to CPU 2 |
| **Varied** | kernel; variant (scalar, autovec, native); $N = 2^8 \dots 2^{25}$ |
| **Repetitions** | 12 per point; median, IQR error bars |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh baseline` |

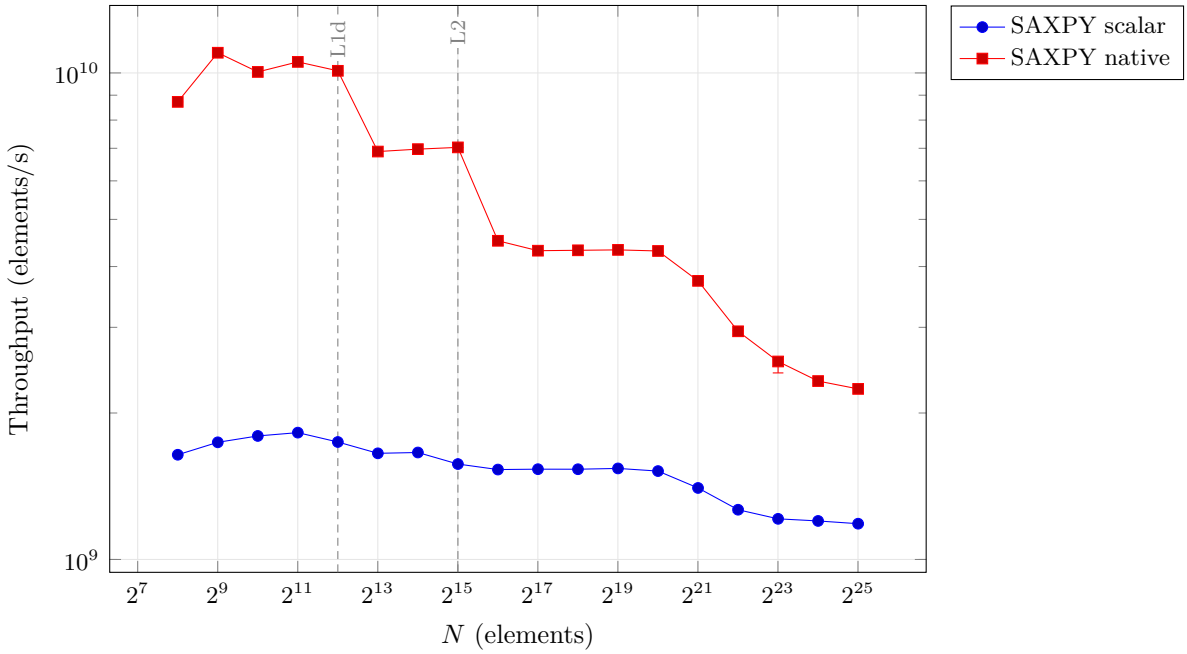



Figure 2: Achieved SAXPY throughput (elements/s, median and IQR). Native drops to a DRAM-limited plateau for large $N$, 1.89–1.94× above scalar, which loses comparatively little outside cache.

### 5.1 Analysis

At $N = 4096$ (L1-resident) the `native` speedups are SAXPY 5.79× (20.2 GFLOP/s), stencil 3.49× (17.3 GFLOP/s), elementwise multiply 3.55× and dot 1.01×. `autovec` reaches about half of `native` for SAXPY (3.02×) and 70 % to 80 % of it for multiply and stencil (2.54×, 2.79×). None reaches the full 8× lane count even in cache: each vector FMA still needs two 256 bit loads and a store, and native SAXPY needs 0.22 to 0.28 cycles per element in L1 (Section 6), against the 0.125 cycles/element (one 8-lane iteration per cycle) that two load ports and one store port would allow. So the loop is limited by load/store issue well before it is FLOP-bound.

The kernels mostly behave as predicted, and the exceptions are informative. *Elementwise multiply* streams three arrays (12 B/element), so it leaves L1 earlier (near $N \approx 2700$) than SAXPY. Its speedup peaks at 5.54× at $N = 2048$ and has already dropped to 3.55× at $N = 4096$. The *stencil* has the highest FLOP count (5) and reuses $x_{i-1}, x_i, x_{i+1}$, so it stays the most compute-bound and keeps a 2.6–2.7× advantage even in DRAM (Figure 1). Its native speedup is irregular, though: 3.3–3.5× for $N = 1024 \ldots 16\,384$ but 4.5–6.2× for $N = 32\,768 \ldots 1\,048\,576$. In other words, the cache-resident stencil runs *slower* per element than the L3-resident one (Section 12). The *dot product* is the reduction case from Section 4: packed multiplies, serial sum, ≈ 1× at every size, at 1.33 GFLOP/s to 1.49 GFLOP/s.

By $N = 16\,777\,216$ (128 MiB, far past L3) every speedup has compressed: SAXPY 1.94×, multiply 1.51×, stencil 2.72×, dot 1.00× (GFLOP/s of every kernel and build: Table 5). Figure 2 shows the asymmetry behind this. Native SAXPY falls from 20.2 to 4.7 GFLOP/s on a DRAM plateau, while scalar only falls from about 3.5 GFLOP/s in L1 to 2.4 GFLOP/s in DRAM. The two plateaus do not merge because scalar SAXPY stays largely instruction-bound even in DRAM. It needs 1.5 to 1.8 cycles of core work per element with all data in cache, at an IPC near 4, and 2.3 cycles per element from DRAM, still at an IPC of 3.0 (Section 6). At the fixed 2.8 GHz clock this persistent ratio is larger than in the turbo-clock sweep (1.32–1.44×, Section 2), as expected if

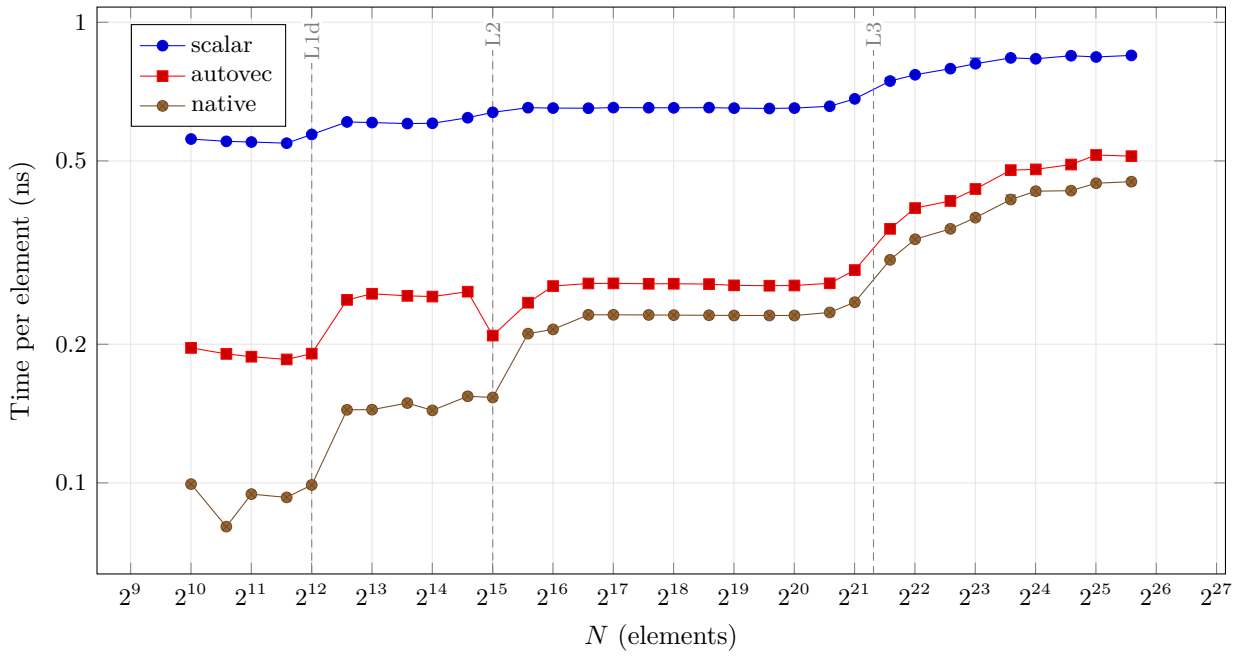



Figure 3: SAXPY time per element across cache levels. The native curve rises in steps at each cache boundary; in DRAM it stays 1.88–1.96× below the scalar curve.

scalar is core-bound and native memory-bound. Separate sweeps agree: SAXPY at $N = 16\,777\,216$ gives 1.94× (baseline and locality), 1.95× (negative case), 1.99× (data type) and 1.91–2.02× across the alignment sweep's offsets, and 1.88–1.92× at $N = 33\,554\,432$. The curves bend downward at the L1 ($N \approx 4096$) and L2 ($N \approx 32\,768$) boundaries; Section 6 resolves those transitions and Section 10 ties each plateau to a measured bandwidth roof.

## 6 Locality Sweep

**Hypothesis (from first principles, then compared with the data)**

For float32 SAXPY (two arrays, 8 B/element) the per-element time should be flat and low while the working set fits in L1 ($2N \cdot 4\,\text{B} \leq 32\,\text{KiB}$, i.e. $N \lesssim 4096$), step up at the L2 boundary ($N \approx 32\,768$) and again past L3 ($N \approx 2.6 \times 10^6$), and saturate in DRAM. The scalar and SIMD curves should be far apart in L1 and converge once DRAM bandwidth, which both share, dominates.

| | |
|---|---|
| **Fixed** | kernel SAXPY; float32; unit stride; aligned; pinned to CPU 2 |
| **Varied** | $N$ (dense grid, powers of two and midpoints, across L1/L2/LLC/DRAM); variant |
| **Counters** | user-space cycles, instructions, LLC misses and L1D read misses per repetition via `perf_event_open` (Section 2.6); written to `locality_counters.csv` when the counters can be opened |
| **Bandwidth** | effective SAXPY bandwidth = elements/s × 12 B (read $x$, read and write $y$), compared with the independently measured triad roof of each level |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh locality` |

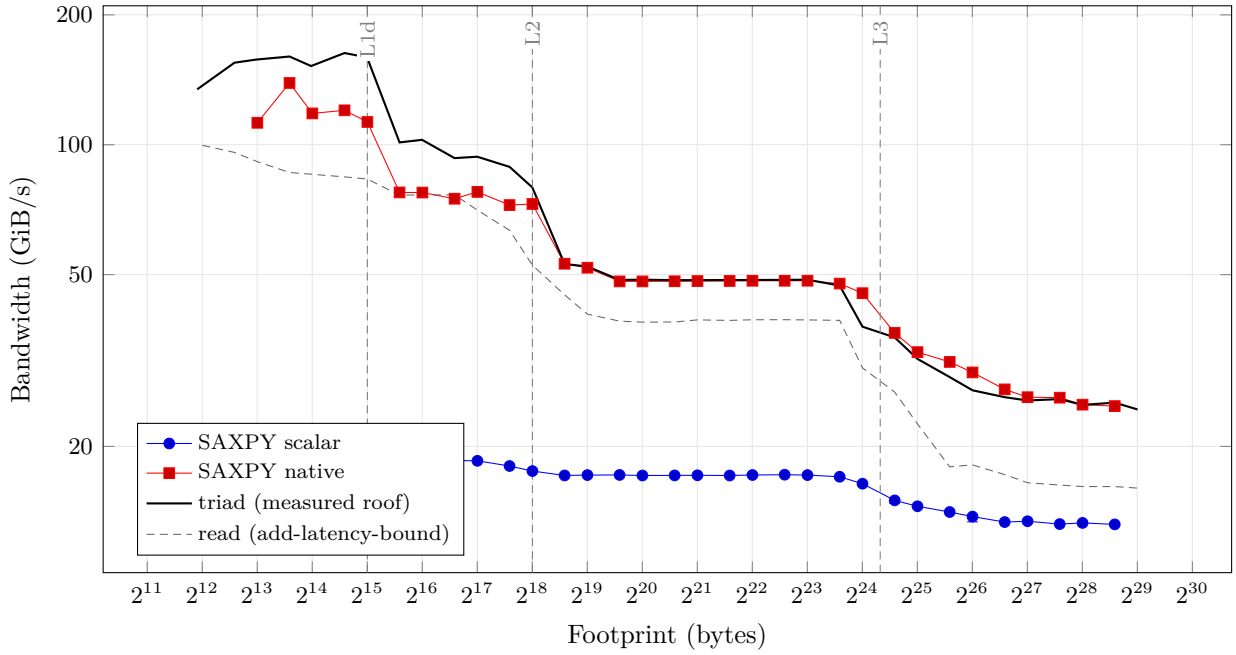
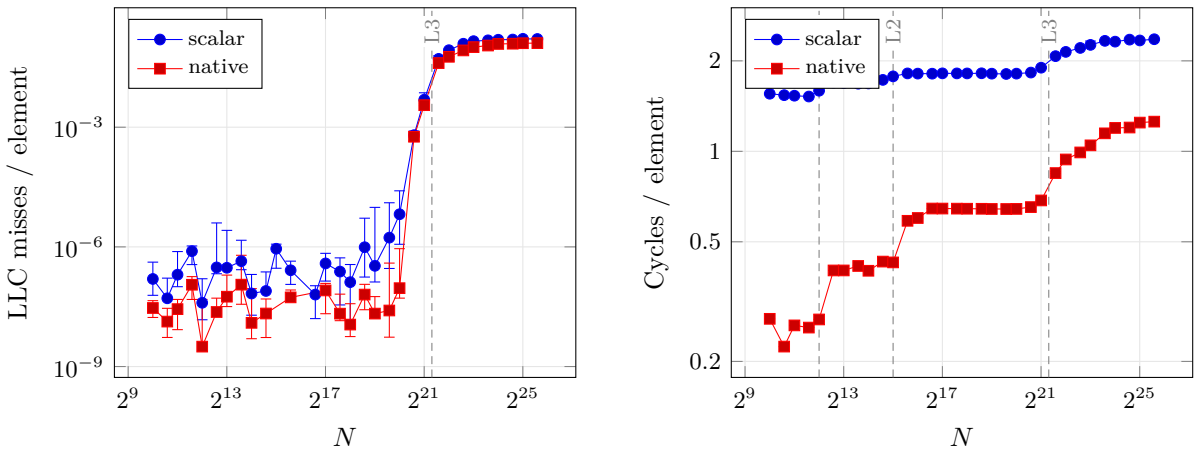



Figure 4: Effective bandwidth versus footprint: SAXPY (from Figure 3) against the independently measured triad (explicit traffic below 32 KiB, incl. write-allocate above). Where the native SAXPY curve meets the triad curve, SIMD SAXPY is bandwidth-bound at that level. The dashed read stream is *not* a bandwidth ceiling: four dependent accumulator chains with a 4-cycle add latency allow at most one 32 B load per cycle (≈ 89 GiB/s), which is what it shows in L1 and L2.




(a) LLC misses per element.

(b) Core cycles per element.

Figure 5: User-space hardware counters for the locality sweep (`perf_event_open`).

Table 6: Effective SAXPY bandwidth (GiB/s, 12 B/element) at one $N$ inside each level, next to the triad roof measured for that level by the independent bandwidth benchmark (actual traffic: explicit loads and stores in L1, plus write-allocate in L2–DRAM, Section 2.6).

| Level | $N$ | Footprint | scalar | autovec | native | triad roof |
|---|---|---|---|---|---|---|
| L1 | 2048 | 16 KiB | 20.3 | 59.5 | 118.2 | 155.6 |
| L2 | 16384 | 128 KiB | 18.5 | 44.0 | 77.8 | 90.5 |
| L3 | 524288 | 4 MiB | 17.2 | 41.6 | 48.4 | 48.6 |
| DRAM | 33554432 | 256 MiB | 13.3 | 21.7 | 25.0 | 24.3 |

### 6.1 Analysis

The measured transitions line up with the cache sizes. While $N \lesssim 4096$ (L1-resident) native SAXPY takes 0.080 ns to 0.100 ns per element, a 5.6–6.9× speedup over scalar's 0.55 ns to 0.57 ns. Past the 32 KiB L1, at $N = 6144$, the native time jumps to 0.144 ns and the speedup drops to 4.2×; it stays at 4.0–4.2× through L2. Past the 256 KiB L2, at $N = 49\,152$, it falls to 3.1× (0.21 ns), and the L3 plateau holds 2.8× (0.23 ns) from $N = 98\,304$ to $1\,572\,864$. Past L3 the gap narrows but does *not* close, contradicting the hypothesis: from $N = 16\,777\,216$ on, native settles at 0.43 ns to 0.45 ns per element and scalar at 0.83 ns to 0.85 ns, a remaining speedup of 1.88–1.96×.

Table 6 and Figure 4 convert the same data to bandwidth. Native SAXPY moves 118 GiB/s in L1, 77.8 in L2, 48.4 in L3 and 25.0 GiB/s from DRAM (at the representative $N$ of the table), and each level boundary is a step in Figure 3. The independently measured triad roofs are 155.6, 90.5, 48.6 and 24.3 GiB/s, so native SAXPY reaches 76, 86, 100 and 103 % of the roof. It is therefore bandwidth-bound in L3 and DRAM, where it runs at the measured roof. In DRAM it slightly exceeds the triad roof, staying below the copy stream's 25.7 GiB/s; copy has the same two-reads-one-write pattern. In L1 and L2 it falls short: there the limit is the core's load/store issue rate (below), not the cache. Scalar SAXPY, by contrast, is flat at 17 GiB/s to 20.5 GiB/s through L1, L2 and L3, far below every cache roof, and drops to 13.3 GiB/s in DRAM.

*Counters* (Figure 5; effective clock 2.78 GHz to 2.79 GHz at every point). LLC misses per element stay below $10^{-5}$ up to an 8 MiB working set ($N = 1\,048\,576$). At 12 and 16 MiB, still inside the 20 MiB L3, they already rise to $\approx 6 \times 10^{-4}$ and $3.6 \times 10^{-3}$ to $4.9 \times 10^{-3}$, plausibly because the shared, not perfectly LRU L3 already loses some lines of a working set close to its capacity. For DRAM-resident native SAXPY they reach 0.122 to 0.128 ($N = 16\,777\,216 \ldots 50\,331\,648$), matching the compulsory $2/16 = 0.125$ (two 64 B lines, $x$ and $y$, per 16 elements; the dirty $y$ line is written back, not missed again). So the vector loop moves no more data than it must. Scalar shows 0.156 to 0.164, about 28 % more. Its cause is open. Untested hypothesis: at scalar's lower request rate the L2 streamer prefetches further ahead of the demand stream, and its prefetch requests, which the generic LLC-miss event may count, include lines evicted before use or fetched past the end of each pass. A per-element IMC read count (see Section 12) would show whether the extra misses are extra DRAM traffic or only extra prefetch requests. Either way, scalar does not lose because the vector loop saves misses. The cycle counts explain the remaining gap:

| cycles/element (IPC) | L1 ($N = 2048$) | L2 ($N = 16\,384$) | L3 ($N = 524\,288$) | DRAM ($N = 33\,554\,432$) |
|---|---|---|---|---|
| scalar | 1.53 (4.6) | 1.68 (4.2) | 1.81 (3.9) | 2.34 (3.0) |
| native | 0.26 (3.1) | 0.40 (1.9) | 0.64 (1.2) | 1.25 (0.6) |

Native SAXPY's cycles per element grow 4.7× from L1 to DRAM and its IPC falls to 0.6: it is waiting on memory. Scalar SAXPY already spends 1.53 cycles per element of pure core work

in L1, at an IPC of 4.6. That is at this core's issue limit (4 fused micro-ops per cycle, with compare-and-branch pairs macro-fused). In DRAM it needs 53 % more (2.34 cycles) but still retires 3.0 instructions per cycle. So most of its DRAM time is instruction execution, not waiting, and $2.34/1.25 = 1.88$ is exactly the measured DRAM speedup at this size. This is the mechanism of the "compression" the rubric asks about. SIMD does not get slower in DRAM; once native SAXPY sits on the bandwidth roof, its advantage shrinks to the scalar loop's (mostly instruction-bound) time divided by the memory time. That ratio depends on the clock: at turbo ($\approx 4.9$ GHz) the same scalar work takes less time while the memory time stays fixed, and that sweep measured only 1.32–1.44× (Section 2). Absolute native time grows step by step while scalar time is nearly flat, which is why the *ratio* shrinks at every level boundary.

# 7 Alignment and Tail Handling

**Hypothesis (from first principles, then compared with the data)**

A start address that is not 32 B-aligned should force the 256 bit loads to be unaligned and/or trigger a scalar peel prologue, costing a modest fixed overhead that is most visible at small $N$. Because this is a client Skylake core, unaligned `vmovups` that happen to stay within a cache line should be nearly free, so we expect the penalty to be small (tens of percent) and to recur with 64 B period as the offset sweeps a cache line.

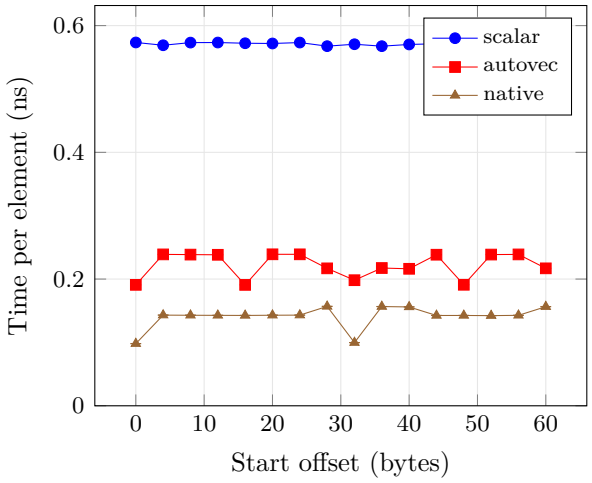
**Hypothesis (from first principles, then compared with the data)**

(For the extended sweep.) The `scalar` build issues 4 B loads that never straddle a line, so it should be insensitive to the offset; `autovec` (16 B loads) should show a smaller penalty than `native`. With a DRAM-resident array the misalignment cost should disappear into the memory stall time for all builds. For short loops, the time per *call* should grow in steps of 8 elements for `native` and 4 for `autovec` (one vector iteration per step), with the vectorized epilogue making the remainder cheap but not free.

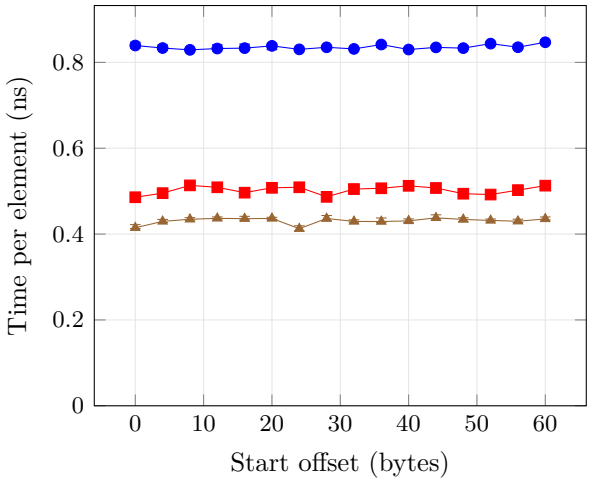
| | |
|---|---|
| **Fixed** | kernel SAXPY; float32; unit stride |
| **Varied** | start offsets $0, 4, 8, \ldots, 60$ B (every multiple of 4) at $N = 4096$ (L1-resident, isolates the prologue from bandwidth) and $N = 16\,777\,216$ (128 MiB, DRAM-resident); tail: every $N$ from 8 to 72 and $N$ stepping across an 8-lane boundary near 4096, offset 0; all three build variants |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh alignment` |

## 7.1 Analysis

*Misaligned starts, L1-resident* (Figure 6a). The penalty is real, it scales with vector width, and it repeats with the load width. Native SAXPY runs at 0.098 ns to 0.100 ns per element at offsets 0 and 32 B (both 32 B-aligned, since the base is 64 B-aligned). Every other offset costs 0.142 ns to 0.143 ns (46 %) or, at 28, 36, 40 and 60, up to 0.157 ns (60 %). `autovec` (16 B loads) is fast exactly at the multiples of 16 (offsets 0, 16, 32, 48: 0.191 ns to 0.198 ns) and 13 % to 25 % slower elsewhere. `scalar` varies by only 1.0 % (0.568 ns to 0.573 ns). This ordering, native 60 % > autovec 25 % > scalar 1 %, is what cache-line splits predict. At a non-multiple-of-32 offset every second 32 B load straddles a 64 B line, but only every fourth 16 B load does, and a 4 B scalar load never does. GCC does not peel the loop to an aligned start (Section 4), so every split load pays. The finer structure is not explained. Both vector builds have a second level at offsets 28, 36, 40 and 60: native is slower there (60 % instead of 46 %), and autovec is faster (13 % instead of 25 %), although counting split

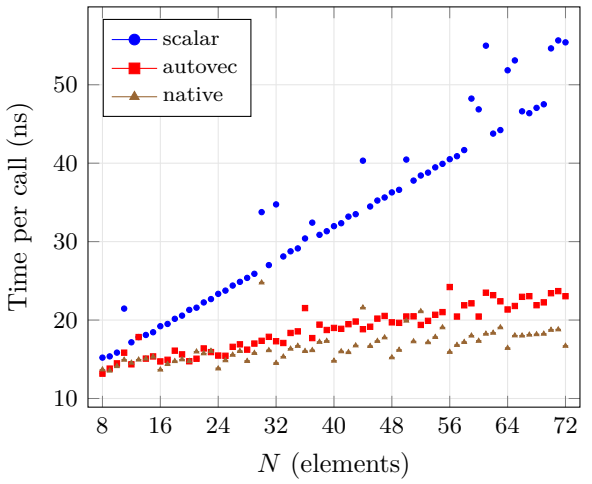



(a) $N = 4096$ (L1-resident).

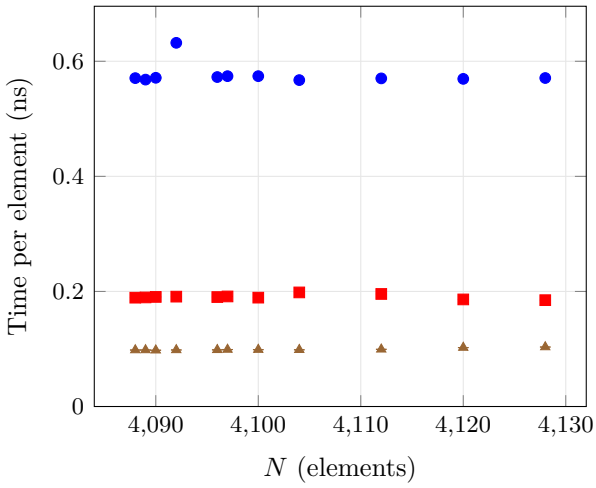



(b) $N = 16\,777\,216$ (DRAM-resident).

Figure 6: Cost of a misaligned start for SAXPY, by build variant (median, IQR error bars).




(a) Short loops: time per call, $N = 8 \ldots 72$ (IQR bars, mostly hidden by the marks).




(b) Across an 8-lane boundary near $N = 4096$.

Figure 7: Tail handling: remainder elements processed by the vectorized epilogue (offset 0).

loads predicts the same number at all non-aligned offsets. Untested hypothesis: an interaction between the split *stores* to `y` and the following loads of `x`, which have the same offset, for example in the store-to-load address check. Offsetting `x` and `y` independently, which our harness does not support yet, would separate the load and store effects. Every configuration's IQR is below 0.3 %.

*DRAM-resident* (Figure 6b). At $N = 16\,777\,216$ the penalty all but disappears. The spread across offsets is 6.1 % for native (0.413 ns to 0.438 ns), 5.7 % for autovec and 2.1 % for scalar, and every IQR is at most 2.9 %. Offset 0 is among the fastest for both vector builds, but beyond that the split-load pattern of L1 does not carry over. Native is equally fast at offset 24, and autovec's multiples of 16 are not consistently faster: offset 32 is among its slower points. So the split-load cost hides under DRAM misses, and what remains is a few percent at most, comparable to process-to-process variation.

*Tails* (Figure 7). Near $N = 4096$ the time per element is flat across the 8-lane boundary (native 0.098 ns to 0.103 ns, autovec 0.185 ns to 0.198 ns, scalar 0.57 ns to 0.63 ns). With 1 to 7 leftover elements out of 4096, the remainder is invisible. Short loops expose it. For $N = 8 \ldots 72$, native's time per call is lowest at multiples of 8 (13.7 ns at $N = 16$ rising steadily to 16.7 ns at $N = 72$, only $\approx 0.05$ ns per element) and highest just below them, at $N \equiv 7 \pmod 8$ typically 2 ns to 3 ns more. That sawtooth is the remainder path from the disassembly (Section 4): one 128 bit step for 4 elements and then up to three scalar iterations. Native also has isolated outliers that the remainder does not explain: 24.8 ns at $N = 30$ (neighbours 15.8 ns to 16.1 ns), 21.6 at $N = 44$, 21.1 at $N = 52$ and 19.9 ns at $N = 50$. `autovec` grows at $\approx 0.15$ ns per element (outliers at $N = 13$, 36 and 56) and `scalar` linearly at $\approx 0.63$ ns per element (outliers at, e.g., $N = 11$, 30, 44, 61). Each outlier is stable within the run (IQR below 1 %); that $N = 30$ and 44 are slow in two builds suggests the per-$N$ array placement rather than the kernel, an untested hypothesis. Every call also pays a fixed $\approx 10$ ns to 13 ns (call, loop setup and the run-time aliasing check of the versioned loop) before the first element. So the per-element speedup of short loops is far below the 5.8× at $N = 4096$: native beats scalar by only 1.11× at $N = 8$ and by 3.32× at $N = 72$. A masked tail (AVX2 `vmaskmovps`; this CPU has no AVX-512 mask registers) would replace the vector step and scalar remainder by one masked iteration; we did not implement one.

# 8 Stride and Gather Access

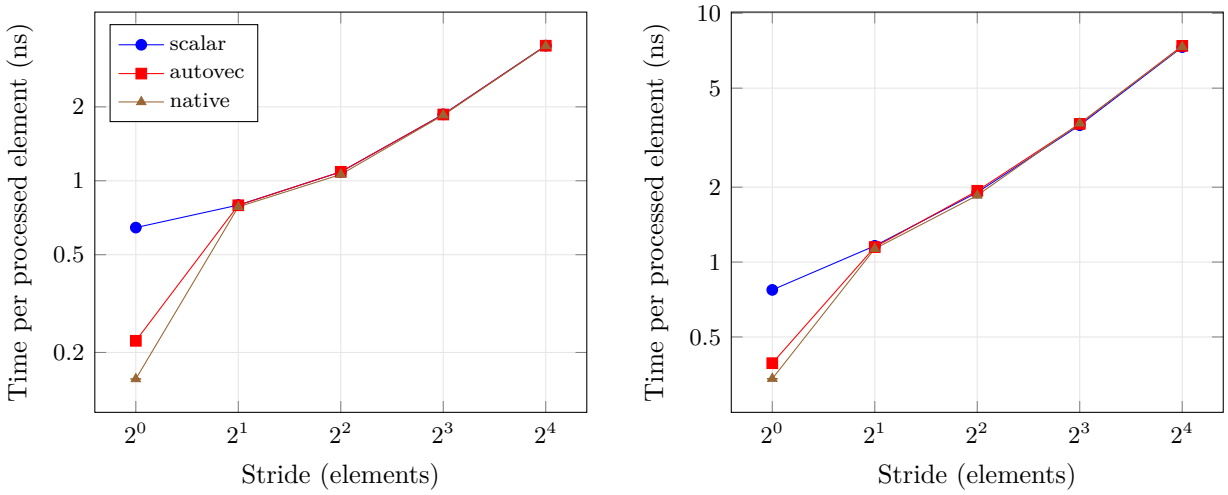
**Hypothesis (from first principles, then compared with the data)**

Unit stride lets each 64 B cache line serve 16 consecutive float32 elements, so SIMD helps. As the stride grows, each vector still needs 8 elements but they come from further apart: once the stride reaches 16 elements (64 B) every element sits in its own cache line and effective bandwidth drops by up to 16×. We therefore expect time per processed element to rise roughly linearly with stride until it saturates at one cache line per element, and we expect the SIMD advantage to vanish for any non-unit stride, because the compiler must gather lanes individually. Gather (random permutation) should be the worst case.

**Hypothesis (from first principles, then compared with the data)**

(For the extended sweep.) The explicit AVX2 gather (`vgatherdps`, Section 4) loads eight lanes per instruction but still needs one cache line per element and eight scalar stores, so we expect it to be at best modestly faster than the scalar gather in cache and no faster in DRAM. On this CPU the "Gather Data Sampling" microcode mitigation is active (`/sys/devices/system/cpu/vulnerabilities/gather_data_sampling`: "Mitigation: Microcode"), which is reported to slow `vgatherdps` considerably, so it may even lose to the scalar loop.

| | |
|---|---|
| **Fixed** | SAXPY; float32 |
| **Varied** | access (unit, stride 1/2/4/8/16, gather, explicit AVX2 gather); variant (scalar, autovec, native; AVX2 gather native only); $N = 32\,768$ processed elements (span 256 KiB × $s$: L2 for $s = 1$, L3 for $s = 16$) and $N = 4\,194\,304$ (DRAM for every $s$) |
| **Definition** | stride $s$ processes elements $0, s, 2s, \ldots$ over an array of $Ns$ elements; gather applies SAXPY through a random permutation index so every element is touched once with no spatial locality. |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh stride` |




(a) $N = 32\,768$ (cache-resident span). (b) $N = 4\,194\,304$ (DRAM-resident).

Figure 8: Effect of stride on SAXPY time per *processed* element (median, IQR error bars). The three builds coincide for every non-unit stride.

Table 7: Time per processed element (ns) by access pattern. *AVX2 gather* is the explicit `vgatherdps` kernel (native build only); speedup is scalar over the faster native kernel (the AVX2 gather has no strided form, hence –).

| $N$ | Access | scalar | autovec | native | AVX2 gather | speedup (×) |
|---|---|---|---|---|---|---|
| 32768 | unit | 0.636 | 0.225 | 0.178 | – | 3.57 |
| 32768 | stride 2 | 0.796 | 0.795 | 0.783 | – | 1.02 |
| 32768 | stride 4 | 1.087 | 1.088 | 1.061 | – | 1.02 |
| 32768 | stride 8 | 1.865 | 1.862 | 1.851 | – | 1.01 |
| 32768 | stride 16 | 3.551 | 3.548 | 3.538 | – | 1.00 |
| 32768 | gather | 2.023 | 2.100 | 2.157 | 3.225 | 0.94 |
| 4194304 | unit | 0.784 | 0.370 | 0.334 | – | 2.35 |
| 4194304 | stride 2 | 1.162 | 1.149 | 1.131 | – | 1.03 |
| 4194304 | stride 4 | 1.905 | 1.934 | 1.852 | – | 1.03 |
| 4194304 | stride 8 | 3.549 | 3.594 | 3.611 | – | 0.98 |
| 4194304 | stride 16 | 7.313 | 7.396 | 7.336 | – | 1.00 |
| 4194304 | gather | 10.205 | 9.979 | 10.198 | 10.911 | 1.00 |

## 8.1 Analysis

Table 7 makes the pattern stark. Only unit stride lets `native` beat `scalar`: 3.6–4.1× at $N = 32\,768$ and 2.35× in DRAM. The range at $N = 32\,768$ comes from two measurements of the same configuration. The table's *unit* row (plain `saxpy_f32`: scalar 0.636, native 0.178 ns, 3.57×) and the stride-1 point of Figure 8a (`saxpy_f32_stride` with $s = 1$: scalar 0.645, native 0.156 ns, 4.13×) run the identical machine code: the scalar loops match instruction for instruction, and the native stride kernel takes the same vectorized path when $s = 1$ (checked in the disassembly). They also use the same allocation. The differences (1 % for scalar, 14 % for native) are therefore process-to-process variation. Untested hypothesis: at this $N$ the two arrays fill exactly the 256 KiB L2, so conflict misses depend on which physical pages each process gets. Repeating the configuration in many

processes, either on 2 MiB huge pages (fixed physical contiguity) or with the physical addresses recorded via `/proc/self/pagemap` (root), would confirm or refute it (Section 12). At stride 2 and beyond the three builds agree within 0.98 to 1.03× (native vs. scalar) at both sizes; `autovec` is within 10 % of them everywhere, as expected since its strided loop is equally scalar. Two things happen at once. First, the effective bandwidth collapses. In DRAM the time per processed element grows from stride 2 to 4 to 8 to 16 as 1.13 → 1.85 → 3.61 → 7.34 ns (native), approaching a doubling per step. Converted to bandwidth (`stride_bw.csv`; useful = 12 B per processed element, fetched = $3 \times \min(4s, 64)$ B: a line of $x$ and of $y$ read and the dirty $y$ line written back), the useful bandwidth falls from 9.9 to 6.0, 3.1 and 1.5 GiB/s, while the fetched line traffic is 19.8, 24.1, 24.8 and 24.4 GiB/s—from stride 4 on, exactly the DRAM triad roof (24.3 GiB/s). The memory system is saturated, but with lines of which only $4s/64$ is used (1/16 at stride 16). With $N = 32\,768$ the span stays in L2/L3 and the progression is milder (0.78 → 1.06 → 1.85 → 3.54 ns): useful 14.3 → 3.2 GiB/s, fetched 28.5 → 42.1 → 48.3 → 50.5 GiB/s, reaching the L3 roof (48.6 GiB/s) at stride 8. The cost per element is set by the cache lines touched, not by arithmetic. Second, the SIMD advantage disappears. The vectorized body exists only for $s = 1$ (GCC's run-time loop versioning, Section 4); every $s > 1$ runs the scalar copy of the loop, so no build has vector lanes to offer. Gather is the worst pattern: 10.2 ns per element in DRAM (native), 31× slower than unit stride, because the random permutation also defeats the hardware prefetcher. Even at $N = 32\,768$ it costs 2.2 ns, 12× unit stride.

*Hardware gather.* The explicit AVX2 kernel takes 3.23 and 10.91 ns per element at $N = 32\,768$ and 4 194 304. That is 1.59× and 1.07× the time of the scalar build's gather, i.e. *slower*, decisively so when the data is cache-resident. The hypothesis above anticipated this. Each 8-lane step issues two `vgatherdps`, which the active Gather Data Sampling microcode mitigation is known to make expensive, followed by eight scalar stores. In DRAM the random line fetches dominate either way and the difference shrinks to 7 %. We could not run without the mitigation (that needs a reboot with root), so the attribution remains a hypothesis (Section 12). Either way, a SIMD gather does not recover the unit-stride advantage. The practical lesson is that access pattern, not vector width, is the first-order knob for these kernels.

# 9 Data Type Comparison

**Hypothesis (from first principles, then compared with the data)**

A 256 bit register holds 8 float32 but only 4 float64 lanes, so float64 should show roughly half the SIMD speedup of float32 for the same kernel while data is cache-resident. int32 elementwise multiply should behave like float32 (8 lanes, one `vpmulld`). In DRAM, float64 moves twice the bytes per element, so its *absolute* throughput should be lower and its speedup should compress toward 1× just as float32 does.

| | |
|---|---|
| **Fixed** | unit stride; aligned |
| **Varied** | data type (float32, float64, int32); kernel; variant; $N = 2^{10} \dots 2^{24}$ |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh dtype` |

## 9.1 Analysis

The lane-count prediction holds for most cache-resident points (Table 8). At $N = 1024$ the native float64 speedup is 0.52 to 0.59 of float32's for all three kernels (multiply 2.82× vs 5.19×, SAXPY 3.28× vs 5.60×, stencil 1.73× vs 3.35×), close to the expected 1/2. The int32 multiply, which

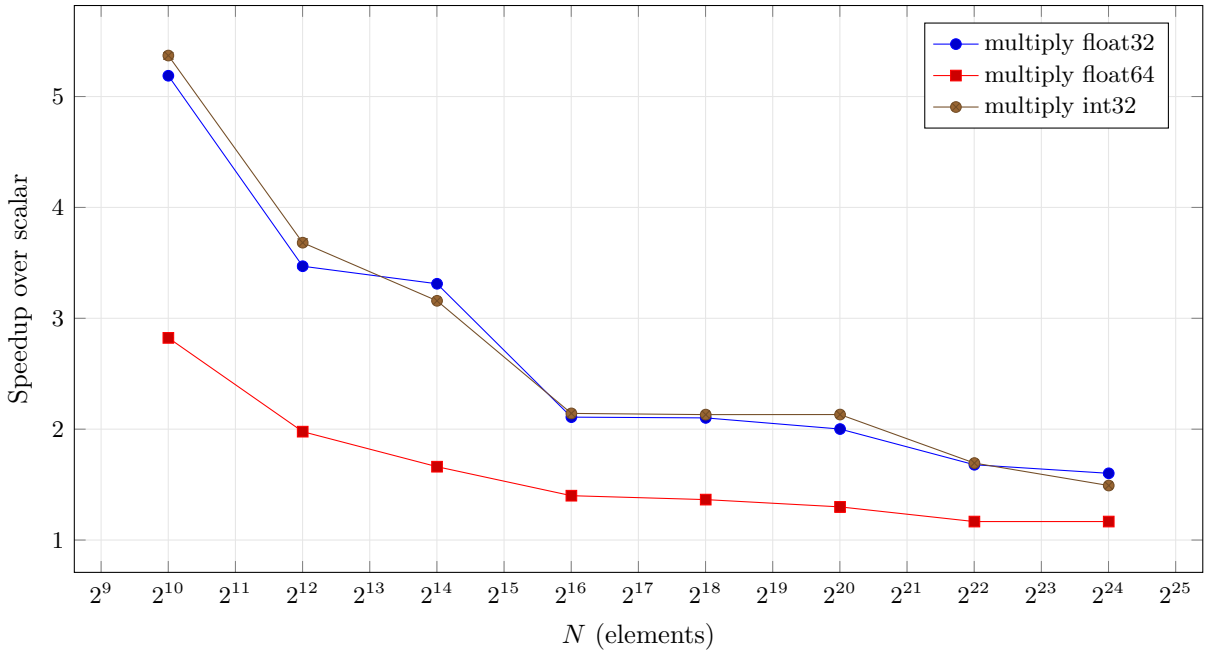



Figure 9: Native speedup of elementwise multiply by data type (median; error bars: range implied by the two IQRs). float32 and int32 (8 lanes) track each other; float64 (4 lanes) is about half. SAXPY and stencil: Table 8.

Table 8: Speedup over scalar by kernel and data type (`dtype.csv`), at $N = 1024$ (L1), 16 384 (L2) and 16 777 216 (DRAM).

| Kernel | Type | autovec, 1024 | native, 1024 | 16 384 | 16 777 216 |
|---|---|---|---|---|---|
| SAXPY | float32 | 2.84 | 5.60 | 4.08 | 1.99 |
| SAXPY | float64 | 1.65 | 3.28 | 2.21 | 1.38 |
| multiply | float32 | 3.53 | 5.19 | 3.31 | 1.60 |
| multiply | float64 | 1.91 | 2.82 | 1.66 | 1.17 |
| multiply | int32 | 1.86 | 5.37 | 3.16 | 1.49 |
| stencil | float32 | 2.75 | 3.35 | 3.44 | 2.79 |
| stencil | float64 | 1.44 | 1.73 | 3.02 | 1.68 |

`native` compiles to one `vpmulld` over 8 lanes, lands on the float32 curve (5.37×). The ratio is not universal, however. At $N = 16\,384$ the stencil keeps 3.02× in float64 against 3.44× in float32 (0.88): the float32 stencil is depressed at exactly these sizes by the slow cache-resident regime of Section 12, while float64 runs at 13.2 GFLOP/s. And by $N = 65\,536$, where multiply is L2/L3-bound, its ratio narrows to 1.40× vs 2.11×.

*Operation cost.* Lane count is not the only factor. `autovec` gains 3.53× on float32 multiply at $N = 1024$ (one `mulps` per 4 lanes) but only 1.86× on int32: SSE2 has no packed 32-bit multiply (`pmulld` is SSE4.1), so the disassembly emulates it with two `pmuludq`, two shifts, two `pshufd` and a `punpckldq`—seven instructions per 4 lanes.

In DRAM every speedup compresses as predicted ($N = 16\,777\,216$, native: float32 1.60–2.79×, float64 1.17–1.68×), and float64 now keeps 60 % to 73 % of the float32 gain rather than half. The separate baseline sweep measured 1.51× for float32 multiply at the same $N$ (Section 5); the difference

Table 9: Measured roofline ceilings, single core (`run.sh ceilings`). Top: register-only accumulator loops (12 independent chains); FLOP/cycle and clock from the user-space cycle counter. Bottom: STREAM-style bandwidth at one footprint per level (GiB/s). "incl. WA" adds the write-allocate read; the roofs in Figure 10 use the explicit triad in L1 (stores hit L1, no write-allocate) and triad incl. WA below it. The read stream (*) is add-latency-bound (Section 2.6) and is not used as a roof.

| Peak loop | GFLOP/s | FLOP/cycle | GHz |
|---|---|---|---|
| scalar_f32 | 5.6 | 1.99 | 2.79 |
| scalar_fma_f32 | 11.2 | 4.00 | 2.79 |
| sse_f32 | 22.3 | 8.00 | 2.79 |
| sse_f64 | 11.1 | 3.98 | 2.79 |
| avx2_fma_f32 | 89.2 | 31.99 | 2.79 |
| avx2_fma_f64 | 44.6 | 16.00 | 2.79 |

| | | GiB/s | | | |
|---|---|---|---|---|---|
| Level | Footprint (B) | read* | copy | triad | triad incl. WA |
| L1 | 16,128 | 85.4 | 142.9 | 155.6 | 207.5 |
| L2 | 130,944 | 71.3 | 67.9 | 67.9 | 90.5 |
| L3 | 8,388,480 | 39.3 | 33.4 | 36.4 | 48.6 |
| DRAM | 536,870,784 | 16.1 | 17.1 | 18.2 | 24.3 |

is mostly the native side (1.72 here vs 1.63 GFLOP/s), a 6 % run-to-run spread of DRAM-bound throughput, so we quote multiply's DRAM speedup as 1.5–1.6×. Float64's *absolute* multiply throughput is under half of float32's (0.80 vs 1.72 GFLOP/s) because each element moves 8 B instead of 4. So data type acts through three mechanisms: lane count and the cost of the vector operation set the cache-resident speedup, while bytes per element set the DRAM throughput.

## 10 Roofline Interpretation

**Hypothesis (from first principles, then compared with the data)**

All four kernels have very low arithmetic intensity (well under 1 FLOP/B), so on a machine whose single-core bandwidth is tens of GB/s and whose peak vector rate is over 100 GFLOP/s they should all sit on the *bandwidth* slope of the roofline when DRAM-resident, i.e. be memory-bound. The stencil, with the highest intensity, should come closest to the compute roof.

**Hypothesis (from first principles, then compared with the data)**

(For the measured ceilings.) Even the L1 bandwidth roof should put the ridge point above 0.5 FLOP/B, so the cache-resident native kernels should also be bounded by bandwidth (or by the load/store ports that define it), not by the FMA units.

Attainable performance at arithmetic intensity $I$ (FLOPs per byte of memory traffic) is bounded by

$$P(I) = \min(P_{\text{peak}},\ B_\ell \cdot I), \tag{1}$$

where $P_{\text{peak}}$ is the core's peak FLOP rate and $B_\ell$ the bandwidth of the memory level $\ell$ the data lives in [1]. We draw one bandwidth roof per level (a cache-aware roofline). Both ceilings are *measured by separate microbenchmarks* (Section 2.6), never derived from the kernels plotted against them.

Table 10: Arithmetic intensity (float32) from the actual traffic per element, the same convention as the triad roof: L2, L3 and DRAM include the write-allocate read of a store to an unread line; in L1 every store hits, so there is none.

| Kernel | $I$, L2–DRAM | $I$, L1 | Traffic per element (L2–DRAM) |
|---|---|---|---|
| SAXPY | 0.167 | 0.167 | 2 FLOP/12 B: read $x$, read $y$, write $y$ ($y$ is read first, so no write-allocate) |
| multiply | 0.063 | 0.083 | 1 FLOP/16 B: read $x, y$, write $z$ + write-allocate of $z$ |
| stencil | 0.417 | 0.625 | 5 FLOP/12 B: read $x$ once (reuse), write $y$ + write-allocate |
| dot product | 0.25 | 0.25 | 2 FLOP/8 B: read $x, y$, no store |
| triad (roof) | 0.125 | 0.167 | 2 FLOP/16 B: read $b, c$, write $a$ + write-allocate |

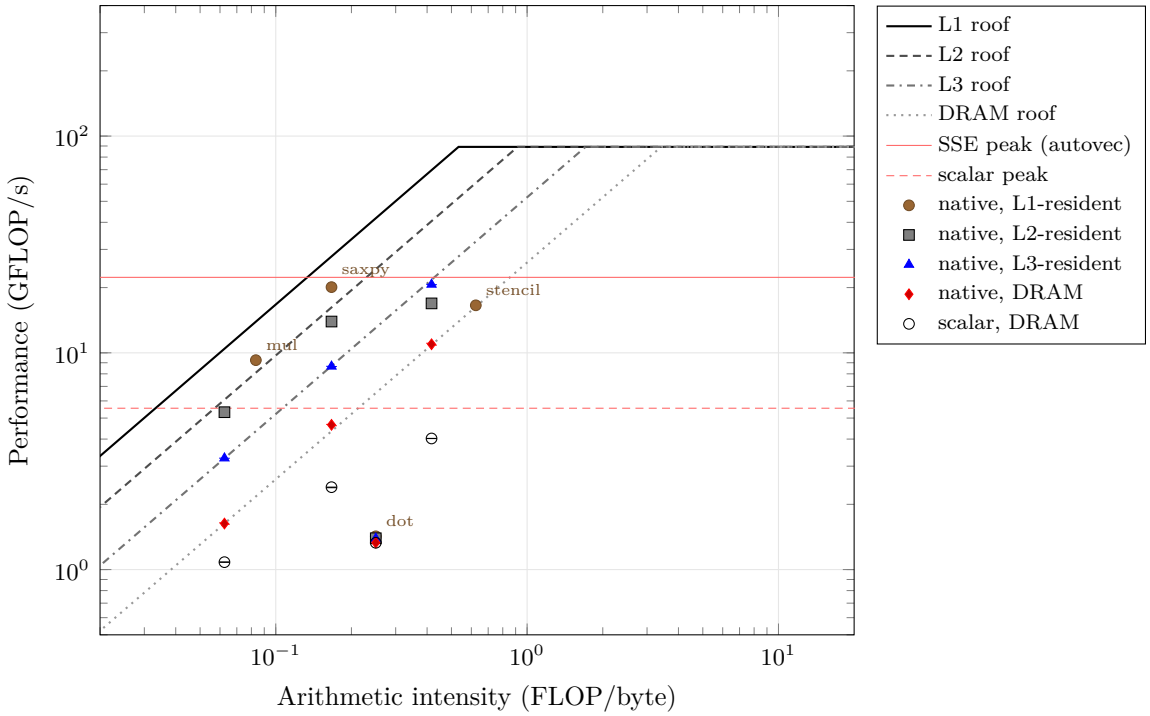



Figure 10: Cache-aware roofline for one core (L1 points at their L1 intensity, Table 10). Roofs: measured triad bandwidth per level, capped by the measured AVX2-FMA float32 peak; thin red lines: the measured SSE and scalar peaks that bound the `autovec` and `scalar` builds. Points: kernel GFLOP/s from Section 5 at $N = 1024$, 16 384, 524 288 and 16 777 216 (L1, L2, L3, DRAM), median with IQR error bars (mostly smaller than the markers).

### 10.1 Analysis

*Ceilings* (Table 9). The AVX2 FMA loop reaches 89.2 GFLOP/s float32 (44.6 float64), which is 31.99 FLOP/cycle at the measured 2.79 GHz. That is exactly the 2 FMA ports × 8 lanes × 2 = 32 FLOP/cycle the core can do, so the ports, not the loop, set this ceiling. The SSE mul+add loop (`autovec`'s ISA, no FMA) reaches 22.3 GFLOP/s (8.00 FLOP/cycle, expected 2 × 4 = 8) and the scalar loop 5.55 GFLOP/s (1.99 FLOP/cycle, expected 2). Triad bandwidth (actual traffic) falls from 155.6 GiB/s in L1 to 90.5, 48.6 and 24.3 GiB/s in L2, L3 and DRAM. In L1 this is the explicit load and store traffic: the triad's stores hit lines already in L1, so the write-allocate read counted below L1 (207 GiB/s incl. WA) does not happen there. The ridge points are $I = P_{\text{peak}}/B_\ell \approx 0.53$, 0.92, 1.71 and 3.42 FLOP/B. The L1 ridge is just above the 0.5 FLOP/B the second hypothesis assumed. Only the stencil, whose L1 traffic is 8 B per element ($I = 0.625$), lies right of it, i.e. nominally under the compute roof in L1.

*DRAM-resident kernels.* Every kernel's intensity (0.06 to 0.42 FLOP/byte) is far left of the DRAM ridge, so all four are bandwidth-bound in DRAM. This is consistent with the hypothesis and with the locality sweep (Section 6). Against the DRAM roof $B_{\text{DRAM}} \cdot I$, native multiply reaches 100 % (1.63 measured vs 1.63 GFLOP/s), the stencil 101 % and SAXPY 107 % (4.65 vs 4.35). Multiply has exactly the triad's access pattern (two reads, one write to an unread line) and lands on the roof. SAXPY, whose pattern differs (two reads, one write to a line it just read), exceeds it by 7 %, and by up to 20 % at the intermediate $N = 8\,388\,608$ (Section 11). The sustainable bandwidth depends a little on the access pattern, so the triad roof is a reference, not an upper bound to the byte. Because the roof comes from a different loop, a kernel landing on it is evidence, not a tautology. The scalar SAXPY, multiply and stencil DRAM points sit at 37 % to 66 % of the same roof (scalar dot: 20 %, latency-bound like native dot). They are the largely instruction-bound loops of Section 6, which is why SIMD still pays 1.51× (multiply), 1.94× (SAXPY) and 2.72× (stencil) at these points.

*Cache-resident kernels.* The L3-resident native points sit at 95 % to 100 % of the L3 roof (SAXPY 100 %, multiply 100 %, stencil 95 %), so L3 bandwidth bounds them exactly. Closer to the core they fall below the roof. In L2, SAXPY reaches 86 % and multiply 88 % of the roof (stencil 42 %). In L1, native SAXPY delivers 20.1 GFLOP/s at $I = 0.167$, 72 % of the L1 roof and 23 % of the FMA peak; multiply reaches 66 % of the L1 roof and the stencil only 19 % of the peak that bounds it there. So cache-resident kernels are bounded neither by the FMA units (confirming the second hypothesis) nor quite by the measured L1 bandwidth. The limit is the loop's own load/store and instruction issue: 0.22 to 0.28 cycles per element for SAXPY versus the 0.125 that two loads and a store per cycle would allow (Section 5). The roofline thus explains the step pattern of Figure 3 from L2 outward, since each step is a move down to a lower roof. Inside L1/L2 it only gives the upper bound.

*The dot product* is the one kernel far *below* every roof: 1.33 GFLOP/s to 1.49 GFLOP/s at all sizes (Section 5). It is not bandwidth-bound but latency-bound by the non-reassociated reduction (Section 4). One scalar `vaddss` per element with a 4-cycle latency gives an in-core ceiling of 2 FLOP/4 cycles = 0.5 FLOP/cycle, i.e. 1.39 GFLOP/s at the measured 2.79 GHz. The measured 1.39 GFLOP/s to 1.42 GFLOP/s for every $N$ from 1024 to 1 048 576 matches it to within 2 %; only $N \leq 512$ runs slightly above it (1.45 GFLOP/s to 1.49 GFLOP/s). Each call starts a new, independent chain, and with short chains the out-of-order core overlaps the end of one call with the next. DRAM runs slightly below it (1.33 GFLOP/s). The roofline, a throughput model, does not apply to the dot product until the dependency chain is broken. To move a streaming kernel toward the compute roof one would have to raise its arithmetic intensity (fuse passes, or block for cache so data is re-read from L1 rather than DRAM); widening the vectors alone cannot help a kernel that is already waiting on memory.

# 11 Negative Example

**Hypothesis (from first principles, then compared with the data)**

A naive benchmark—or a quickly generated one that only ever tests a small array—would conclude that SAXPY enjoys the full $\approx 6\times$ SIMD speedup. We predict instead that the same *vectorized* kernel shows almost no speedup on a DRAM-resident array, because memory bandwidth, not vector throughput, then sets the pace.

| | |
|---|---|
| **Case** | memory-bound SAXPY: a loop that provably vectorizes (`vfmadd213ps` on `ymm`, Section 4) but whose working set far exceeds L3 |
| **Build** | `native` (`-O3 -march=native`) vs `scalar` |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh negative` |

## 11.1 Evidence

The SAXPY loop at `kernels.c:29` is confirmed vectorized at 256 bit with FMA by both the compiler report and the disassembly in Section 4. Yet its speedup over scalar falls with size:

| $N$ | working set | native GFLOP/s | % of roof | speedup |
|---|---|---|---|---|
| 1024 | 8 KiB (L1) | 20.1 | 72 | 5.60× |
| 16 384 | 128 KiB (L2) | 13.7 | 84 | 4.11× |
| 262 144 | 2 MiB (L3) | 8.6 | 99 | 2.82× |
| 4 194 304 | 32 MiB (DRAM) | 6.0 | – | 2.31× |
| 8 388 608 | 64 MiB (DRAM) | 5.2 | 120 | 2.19× |
| 16 777 216 | 128 MiB (DRAM) | 4.7 | 109 | 1.95× |
| 33 554 432 | 256 MiB (DRAM) | 4.5 | 104 | 1.92× |

"% of roof" is native GFLOP/s over $B_\ell \cdot I$ with the measured triad roof of the level the data lives in (Section 10); at 32 MiB the working set straddles L3 and DRAM. The other SAXPY sweeps agree (Section 5).

## 11.2 Explanation

The hypothesis was half right. The $\approx 6\times$ seen in L1 indeed does not survive in DRAM, but the speedup does not fall to "almost none". It settles at 1.92–1.95× in this sweep. Both parts have measured causes. (i) The loop is genuinely vectorized (disassembly), so this is not a failure to use SIMD. (ii) The kernel's arithmetic intensity (0.167 FLOP/B, Table 10) puts it deep in the bandwidth-bound region, and native SAXPY runs at 104 % to 120 % of the DRAM roof $B_{\mathrm{DRAM}} \cdot I = 4.35$ GFLOP/s. $B_{\mathrm{DRAM}}$ is measured by a separate triad benchmark, not by SAXPY. The LLC-miss counter gives 0.122 to 0.128 misses per element for native, the compulsory 2/16 (Section 6). The vector loop is waiting on memory and moving no more data than it must. *No* wider vector, AVX-512 included, could make it faster. (iii) The remaining $\approx 1.9\times$ is not a SIMD gain in the memory-bound sense: scalar SAXPY is still largely instruction-bound in DRAM (2.34 cycles per element at IPC 3.0, versus 1.53 with all data in L1), and the ratio of its cycles to native's (1.25) is the measured speedup. That ratio depends on the clock: with turbo the DRAM-resident speedup was only 1.32–1.44× (Section 2). (iv) The same kernel *does* reach 5.6× when the array fits in L1, so the hardware is capable of the speedup; in DRAM only bandwidth withholds it.

A second, subtler misleading case appears in the dot product (Section 4): its loop reports "vectorized" and the disassembly shows a 256 bit `vmulps`, so a reader who stops at the compiler

report would claim success—yet the serial, non-reassociated reduction leaves the real speedup at $\approx 1\times$. Both cases carry the same lesson for AI-assisted benchmarking: "it compiled to SIMD" is necessary but not sufficient evidence that a kernel is actually faster.

# 12 Anomalies and Limitations

## 12.1 Unexpected results

- **Dot product never speeds up.** We expected a reduction to vectorize at $\approx 8\times$ like the other float32 kernels. Instead native dot stays at $\approx 1\times$ across all sizes. Hypothesis: without `-ffast-math` the compiler may not reassociate floating-point addition, so it vectorizes the multiply but keeps a serial scalar sum. Tested: the disassembly shows `vmulps` on `ymm` followed by a chain of scalar `vaddss` (Section 4), confirming the hypothesis. We chose not to enable `-ffast-math` so that all variants remain bit-reproducible against the reference.

- **Native SAXPY speedup varies within L1.** It is 5.6–5.9× at $N = 1024$, 2048, 3072 and 4096, but 6.86× at $N = 1536$ (locality sweep) and 6.32× at $N = 512$ (baseline). Scalar time is flat across these sizes and L1D misses stay below 0.0011 per element below $N = 4096$; the peak is native needing 0.22 instead of 0.26 to 0.28 cycles per element. Per-call overhead cannot explain it, since it *lowers* small-$N$ speedups (Section 7). Untested hypothesis: the relative placement of `x` and `y` (4K aliasing, as for the stencil below), testable with the same distance sweep.

- **Stencil keeps a** 2.6–2.8× **advantage in DRAM** while SAXPY and multiply fall to 1.88–2.02× and 1.5–1.6×. Its native build is just as bandwidth-bound as theirs: it runs at 101 % of the DRAM roof (Section 10). Its scalar build, however, needs 5 FLOP per element of serial `mulss`/`addss` work and reaches only 4.0 GFLOP/s to 5.0 GFLOP/s at every size (4.0 GFLOP/s, 37 % of the roof, in DRAM). The retained ratio is again scalar's instruction cost, as for SAXPY (Section 6), just larger.

- **Cache-resident stencil is slower than L3-resident stencil.** Native stencil delivers 16.6 GFLOP/s to 17.3 GFLOP/s for $N = 1024 \ldots 16\,384$ but 19.9 GFLOP/s to 20.7 GFLOP/s for $N = 65\,536 \ldots 1\,048\,576$, and 31.4 and 27.4 GFLOP/s at the isolated sizes $N = 512$ and 32 768. The pattern reproduces exactly across separate runs; scalar stencil stays at 4.0 GFLOP/s to 5.0 GFLOP/s. A cache effect would make the rate fall monotonically with $N$; this does not. Hypothesis (untested): 4K aliasing. The small `x` and `y` allocations sit at addresses whose difference makes a store to `y[i]` look like it conflicts with the load of `x[i+1]` in the low 12 address bits, which stalls the load; large allocations come from fresh pages with a different relative offset. It would be confirmed by a sweep of the `x`–`y` distance modulo 4 KiB at fixed $N$, showing the slow rate only at small distances, together with the core's 4K-alias counter (`ld_blocks_partial.address_alias`). We ran neither.

- **Scalar and SIMD SAXPY do not converge in DRAM.** We expected one shared bandwidth plateau; native stays 1.88–2.02× ahead because scalar SAXPY is still largely instruction-bound there (2.34 cycles per element at IPC 3.0) while native sits on the roof (IPC 0.6); the cycle ratio, 1.88, is the measured speedup (Section 6).

- **Same code, different process,** $N = 32\,768$**.** Scalar SAXPY is stable across processes elsewhere (0.650 ns to 0.653 ns per element at all 9 locality sizes from $N = 65\,536$ to $1\,048\,576$), but where the two arrays exactly fill the 256 KiB L2 two measurements of the same machine

code differ by 1 % (scalar) and 14 % (native); page placement is the untested hypothesis (Section 8).

- **Hardware gather loses to scalar.** The explicit `vgatherdps` kernel takes 3.23 ns per element at $N = 32\,768$ versus 2.16 (native scalar loop) and 2.02 ns (scalar build), i.e. 1.5–1.6× *slower*, and it is 7 % slower in DRAM. Hypothesis: the Gather Data Sampling microcode mitigation, which this CPU runs (Section 8) and which is known to make `vgatherdps` much slower, plus the eight scalar stores that AVX2 cannot vectorize. Confirming it would need a boot without the mitigation, which requires root.

### 12.2 Limitations

- **Core-PMU counters only.** The SIMD locality counters are core-PMU, user-space events of the benchmark thread (Section 2.6): cycles, instructions, generic LLC misses and L1D read misses. We did not count DRAM traffic for SIMD. The machine can do it: the companion Cache & Memory report, run on the same machine the same day with `perf_event_paranoid` = 0, reads the memory-controller (`uncore_imc`) counters and finds IMC traffic matching its streaming estimate to within about 1 %. TLB events were not recorded either. The generic LLC-miss event's treatment of hardware-prefetch requests is model-specific, so we use misses per element as a relative measure between builds rather than as exact DRAM traffic. Counting IMC bytes per element for scalar and native SAXPY would turn the roofline's traffic assumption into a measurement.

- **Fixed clock.** All reported experiments were run on 2026-10-21 with governor performance, turbo disabled and SMT off; the cycle counter confirms 2.78 GHz to 2.79 GHz for every locality and ceilings configuration (Section 2). The fixed clock makes the results reproducible, but it is not how the CPU normally runs. With turbo the compute-bound numbers scale up by roughly the clock ratio (1.72–1.75× in our turbo sweep) while DRAM bandwidth does not, so the DRAM-resident SAXPY speedups reported here (1.88–2.02×) are larger than at turbo (1.32–1.44×, Section 2).

- **One machine, one compiler.** All results are from a single i9-10900F with GCC gcc (GCC) 16.2.1 20260810. Clang was used only to confirm the build compiles. No AVX-512 part was available, so the 512 bit regime is out of scope.

- **THP always-on; no core isolation.** Transparent huge pages are `always`, and the pinned core is not isolated from the scheduler (no `isolcpus`). Neither is controlled for; the lock file only serializes cooperating jobs, so the shared-core checks of Section 2 (repetition spread $\leq 1.36\times$, apparent clock 2.78 GHz to 2.79 GHz) are what guard the reported data.

- **Discarded sessions.** Earlier sessions the same day were discarded: those before the GPE storm was masked (Section 2), and a first clean session during which another agent's unlocked smoke tests briefly shared CPU 2 ($\approx 2\times$ slowdowns in some repetitions).

- **Report build backend.** Without `biber`, `biblatex` uses the BibTeX backend, so the single reference is sorted by name/year to avoid a spurious rerun warning.

## 13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

**Tools used**

Claude Code, an AI coding agent, for coding, debugging, running the experiments and drafting this report. All sessions up to 2026-10-20 used the model Claude Opus 5. On 2026-09-14 it produced an unrequested complete implementation, which was rejected and removed the next day. On 2026-09-15 it produced the templates (folder structure, READMEs, Makefiles, files with placeholder markers, report skeletons) that every later step filled in. On 2026-10-12 to 2026-10-20 it produced the implementation and the first report. The 2026-10-21 rework (measured ceilings, counters, AVX2 gather, extended sweeps, the fixed-clock re-run and this revision) used Claude Opus 5.5. GCC gcc (GCC) 16.2.1 20260810 and `objdump` produced the builds and the vectorization evidence.

**Who ran the measurements**

The timed SIMD experiments of 2026-10-21 (the reported runs, plus earlier sessions that were discarded for interference) were executed by Claude Code on the user's machine, with the user's approval. Before the runs the user set the governor to `performance`, disabled turbo and SMT, and masked the ACPI GPE 0x42 interrupt storm. Claude Code checked these settings before each run and serialized its runs with other benchmark jobs through a lock file. All measurements come from these real runs on the machine in Table 1.

**What they helped generate**

The four kernels and their references; the build-variant Makefile; the benchmark driver and the shared timer/allocation/pinning helpers; the experiment scripts (`run.sh`), the summarizer (`summarize.c`) and the vectorization-evidence script; the roofline-ceiling microbenchmarks (`ceilings.c`), the `perf_event_open` counter helper (`perfctr.c`), the explicit AVX2 gather kernel and the post-processing scripts (`speedup_iqr.awk`, `roofline.sh`, `tables.sh`); and the first drafts of these sections.

**Who verified what**

The AI ran the checks: `make test` (578 cases in total), `make report`, the disassembly comparisons, the per-run settings and shared-core checks, and the CSV-to-text spot checks of every quoted number. The user approved each timed session, applied the root-only system settings, and reviewed the report through returned grading feedback, which drove several of the corrections below. "Independently" therefore means checked against measurements and tool output, not against a second human analysis.

**What was independently modified, tested, or rejected**

Concrete corrections, among others. (1) The generated Makefile compiled the *whole* program with each variant's flags; this was restructured so only the kernel translation unit varies, keeping the timing harness identical across builds. (2) The processed-CSV column names and one figure filter had drifted from what the plots expect—the stride figure asked for `n=16777216` while the data was measured at `n=4194304`—which would have produced a silently empty plot; the filter was corrected to the measured size. (3) *An AI-written script was simply wrong*: the `awk` in `verify_vectorization.sh` took its output path as a trailing `VAR=value` argument, which `awk` does not assign until it reaches that argument, so the `BEGIN` block saw an empty string and aborted with a null-redirection error. It was caught the first time `make report` was run and fixed by passing the path with `-v`. We also chose *not* to enable `-ffast-math`, rejecting the easy way to make the dot product vectorize, because it would break bit-reproducibility against the reference—and because the non-reassociated reduction is itself a result worth reporting (Sections 4 and 11).

**Validation**

Correctness: 578 reference and known-answer cases in total (190 scalar, 190 autovec, 198 native) pass (Section 3). Vectorization: confirmed by compiler report and disassembly, not assumed (Section 4). Numbers: every number from the 2026-10-21 runs traces to a committed file (Section A); the memory-bound claims are cross-checked against bandwidth and peak FLOP rate measured by separate microbenchmarks (`src/ceilings.c`), not derived from the kernels themselves (Section 10). Review corrections that changed results: an early roofline bandwidth was derived from SAXPY itself (circular; replaced by the independent ceilings); the AI-drafted claims that scalar and SIMD "converge" in DRAM and that small-$N$ speedups are inflated by per-call overhead were both contradicted by the fixed-clock data and removed; the alignment sweep had silently skipped offsets 40 B to 56 B and was re-run; and the L1 roof first included a write-allocate read that L1-resident stores do not incur.

# 14 Conclusion

On this AVX2 machine SIMD pays off handsomely only when two conditions hold at once: the data fits in cache and the kernel is not a serial reduction. Under those conditions `-march=native` gives SAXPY, elementwise multiply and the stencil roughly 3.5–5.8× over scalar at $N = 4096$ (the generic SSE build gets 2.5–3.0×), below the 8× lane count because load/store issue, not the FMA units, limits the vector loop. Once the working set spills to DRAM the vector builds run *at* the independently measured bandwidth roof, and the advantage compresses to 1.88–2.02× for SAXPY, 1.5–1.6× for multiply and 2.6–2.8× for the stencil. What remains is not extra SIMD throughput but the scalar loops' own instruction cost, which at the fixed 2.8 GHz clock still exceeds the memory time; it would shrink further at a higher clock. Data type scales the cache-resident gain through lane count (float64 usually keeps about half of it, though the stencil at $N = 16\,384$ keeps 3.02× versus 3.44×) and through operation cost (SSE2 int32 multiply: 1.86× versus 3.53× for float32), and access pattern dominates everything else: any non-unit stride or a gather erases the SIMD benefit outright, and AVX2's hardware gather is even slower than the scalar loop on this mitigated CPU. The single most important lesson for writing and benchmarking vectorized code is that "the loop vectorized" is not "the loop is fast". Our memory-bound SAXPY gains nothing from more lanes, and our non-reassociated dot product, though it compiles to genuine SIMD, runs at exactly the 0.5 FLOP/cycle its dependency chain allows. Only a controlled sweep across working-set size, plus disassembly, hardware counters and a measured roofline, reveals why.

# References


[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785


# A Reproducibility

Code, scripts, raw data and this report live in one repository: `ACS-Code/` holds the code and data (this project in `ACS-Code/SIMD_Advantage_Profiling/`, shared helpers in `ACS-Code/common/` and `ACS-Code/tools/`), `ACS-Latex/` the reports (this one in `ACS-Latex/SIMD_Advantage_Profiling/`). The timed runs (2026-10-21, 20:38–20:54) predate their commit; they were measured with the tree committed as `b00db66`, and the measurement code (kernels, timing harness, ceilings, `run.sh`) is

unchanged since, except an opt-in error print in the validation path of `main.c` that is inactive in timed runs. Later commits changed only post-processing scripts, the correctness log and this text. This report corresponds to commit `e7907afb9289`. From the repository root:

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout e7907afb9289
cd ACS-Code
make tools                                  # tools/summarize
make -C SIMD_Advantage_Profiling            # scalar/autovec/native simd_bench + ceilings
make -C SIMD_Advantage_Profiling test       # 578 correctness cases (max error logged)
make -C SIMD_Advantage_Profiling report     # vectorization evidence
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
SIMD_Advantage_Profiling/scripts/run.sh all     # ceilings, baseline, locality, alignment,
                                                # stride, dtype, negative
scripts/sync_data.sh SIMD_Advantage_Profiling   # -> ACS-Latex/SIMD_Advantage_Profiling/data/
cd ../ACS-Latex && make simd                    # -> SIMD_Advantage_Profiling.pdf
```

Before timing, as root: governor `performance`, turbo off (`intel_pstate/no_turbo` = 1), SMT off, and on this machine the ACPI event storm masked with `echo disable > /sys/firmware/acpi/interrupts/gpe42`. None of these settings persists across a reboot (Section 2). The SIMD hardware counters (core-PMU, user-space events) need no root as long as `perf_event_paranoid` $\leq 2$; with `COUNTERS=0`, or where they cannot be opened, `run.sh` skips the counter outputs. `run.sh derive` recomputes the derived tables (roofline points and roofs, per-level bandwidth, stride, data-type and GFLOP/s summaries) from the processed CSVs without measuring.

Table 11: Raw data behind each figure (directories under `ACS-Code/SIMD_Advantage_Profiling/data/raw/`).

| Figure / table | Raw run directory |
|---|---|
| Figures 1 and 2 | `baseline/20261021-203854` |
| Figures 3 to 5, Table 6 | `locality/20261021-204801` |
| Figures 6 and 7 | `alignment/20261021-204858` (`raw.csv`, `raw_tail.csv`) |
| Figure 8, Table 7 | `stride/20261021-205143` |
| Figure 9, Table 8 | `dtype/20261021-205209` |
| Table 5 | `baseline/20261021-203854` |
| Correctness result (Section 3) | `validation/20261021-231629` (`make_test.txt`) |
| Figure 10, Table 9 | `ceilings/20261021-204708`, `baseline/20261021-203854` |
| Negative-case table (Section 11) | `negative/20261021-205359` |
| Table 4 | `vectorization/20261021-001032` |

Each timed-run raw directory holds `raw.csv` (one row per repetition, with full configuration metadata) and `env.txt` (a snapshot of `collect_env.sh`); the processed CSVs plotted above are produced from them by `tools/summarize` plus this project's post-processing scripts (`speedup_iqr.awk`, `roofline.sh`, `tables.sh`) and copied into this report's `data/` by `scripts/sync_data.sh`. Every number from the 2026-10-21 runs traces to a committed file; the turbo-clock comparison figures trace to commit `73f8fb8` (Section 2).